AI SYSTEMS ENGINEERING · WEEK 6

Retrieval-Grounded Assistants

A question, a corpus of documents, and an answer with its sources. The architecture, the data, and the measurement.

Tonight

1. **Failure of the Week** — an airline's chatbot invented a refund policy
2. **Retrieval-grounded assistants** — an internal policy assistant: its architecture, its data, its measurement
3. **Design Studio** — design the airline's disruption assistant
4. **Lab** — build the runbook index: lexical, then vector, then hybrid; recall at 5 on labelled questions; validate a judge against your own labels

FAILURE OF THE WEEK

The bereavement fare

- November 2022. A customer flying to his grandmother's funeral asks the airline's chatbot about **bereavement fares**.
- The chatbot: book now, **apply for the reduced fare within 90 days**.
- The airline's policy page says the opposite: **no reduced fare after travel**. The reply linked to that page.
- He applies. The airline refuses. He goes to a small-claims tribunal.

C$812

damages, interest and fees the tribunal ordered the airline to pay, February 2024

Source: Moffatt v. Air Canada, 2024 BCCRT 149, Civil Resolution Tribunal of British Columbia, 14 February 2024.

FAILURE OF THE WEEK

Where did the answer come from?

From the model, not from the page.

The system had the policy. It retrieved the right page well enough to link it. Then the model wrote a sentence about the policy, and the sentence was not what the page said.

Nothing between the model and the customer compared the two. The customer did, three months later, by reading the page.

Three facts about the system:

- The **answer was not checked against its source**. The citation was decoration; the claim was free.
- **Nobody measured the rate** at which answers agreed with the pages they cited. The first measurement was a tribunal.
- The airline argued the chatbot was **"a separate legal entity responsible for its own actions."** The tribunal called that remarkable, and held the airline responsible for everything on its site.

HOLD THIS THOUGHT

What it cost

- C$812, and a published decision that every article about chatbot liability now cites
- the chatbot was gone from the site by the time the decision was published
- the rule the tribunal wrote down: a company is responsible for what its website says, however the words were produced

What a fix would have needed:

- a **check** that every claim in the answer is supported by the passage it cites, before the answer is shown
- a **measured rate** of unsupported claims, per slice, with a floor that stops a release
- a path for **"I cannot answer this; here is the page"**

Each is a component. None is a model.

Where we are

#	STEP	SO FAR	TONIGHT
1	Scope & requirements	requirements as rates the three numbers	an assistant's requirements: grounded, correct, says when it cannot
2	Frame the AI task	the model's contract: inputs to outputs	an answer with citations, from the passages given, or "not found"
3	Metrics	rates with intervals · the noise floor comparing versions · judges precision at *k* · the holdout recall at *k* · NDCG · the A/B test	recall at *k* against labelled passages groundedness the judge, in full: validated and corrected
4	Data	the golden set · the tables · the point-in-time join the impression log · implicit labels position bias · the index as data	the corpus as a dataset the context as data how labels for text are made
5	Architecture	placement · the queue · the cascade the batch and online scorer the index · two towers · the filters	the indexing path and the request path chunking · the lexical index · hybrid retrieval the reranker · the context
6	Deep dive		
7	Failure, scale & operations	the failure path · a failed run a stale index · the feedback loop	nothing found · a document changed degrading to search

AN INTERNAL POLICY ASSISTANT

Retrieval-Grounded Assistants

An answer to an employee's question, written from the company's own documents, with its sources.

An internal policy assistant

A company with 40,000 employees in 30 countries and 200,000 internal documents: HR policies, IT guides, expense rules, engineering wiki pages. 1,500 of them change every day.

One request: an employee types a question. The system returns **an answer, with the documents it came from**.

Two things make this hard:

- the answer is **in the documents**, and the model has never read them
- the employee is **waiting**, and will act on the answer

THE NUMBERS	
documents	200,000; 1,500 change a day
employees	40,000, in 30 countries
questions	20,000 a day; 5 a second at the peak
the answer	a paragraph with its sources, within 5 s
restricted	a third of the documents, by country, team or role
ground truth	whether the answer was right: a reviewer, or a complaint, days later; for most answers, never

The interview versions of this system: "design a customer-support assistant over the help centre"; "design search over the company's documents, with answers"; "design a legal research assistant."

Elements of designing this system

For this system, as for every system:

- **Architecture.** Where the models run, what surrounds them, what runs when they fail.
- **Data handling.** Where the dataset comes from, how it is kept, what it specifies.
- **Measurement.** How you know the answers are any good.

The models are **treated as given**: inputs to outputs, with a datasheet. Everything else is what we build.

The architecture

What shapes the design

- The answer is **in the documents**. The model has never seen them, and 1,500 of them change every day.
- There are **200,000 documents** and the answer needs **a few passages**. The work is finding them.
- The employee is **waiting**: a paragraph within 5 seconds.
- **Not everyone may read everything.** A third of the documents are restricted.
- The truth arrives **slowly, or never**. Nobody tells the system its answer was wrong, and the employee acts on it.

The obvious design: ask the model



The model's knowledge ends at a date and never included our documents. Whatever it says about our policy, it did not read it.

ASKING THE MODEL ALONE	
what it gets right	fluent, fast, cheap; one call, no infrastructure; fine for what is public and old
what it cannot do	know our policy, which it never read; know a change from this morning; say where an answer came from; tell our Germany rule from another company's

The obvious design, second attempt: give it every document



The corpus does not fit; and at a price per token, every question that carried all of it would cost about twelve dollars and take minutes.

EVERY DOCUMENT IN THE PROMPT	
what it gets right	the answer is in what the model was given; nothing is stale if the corpus is read at the request
what it cannot do	fit: the corpus is a hundred times the largest context window; cost: a price per token, per question; permissions: every employee would be handed every document

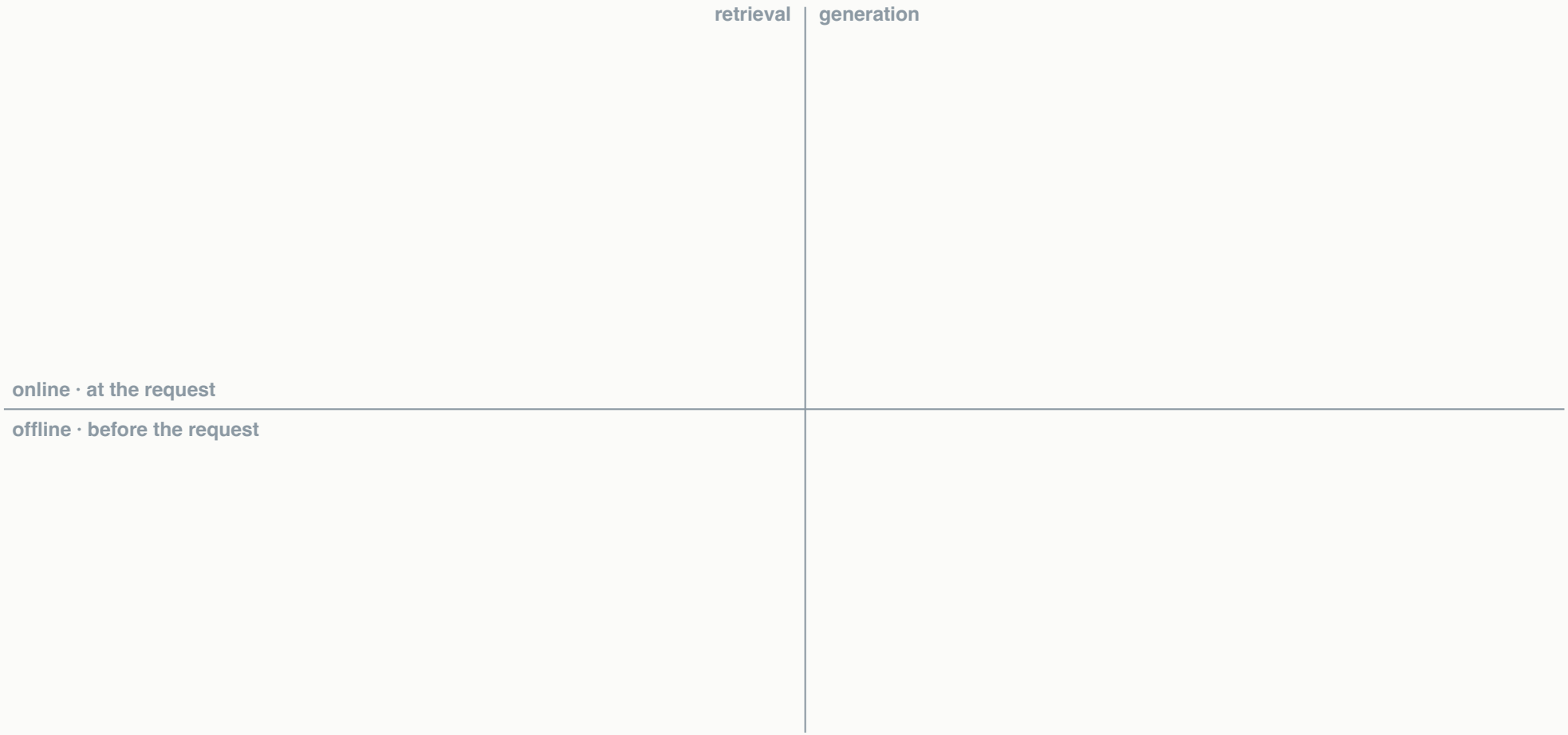
What does the model need to see to answer this question?

Resolution: find the few passages that answer the question, and give the model those

Index every document before any question arrives. At the request, retrieve the passages that answer this question, and let the model write from them alone.

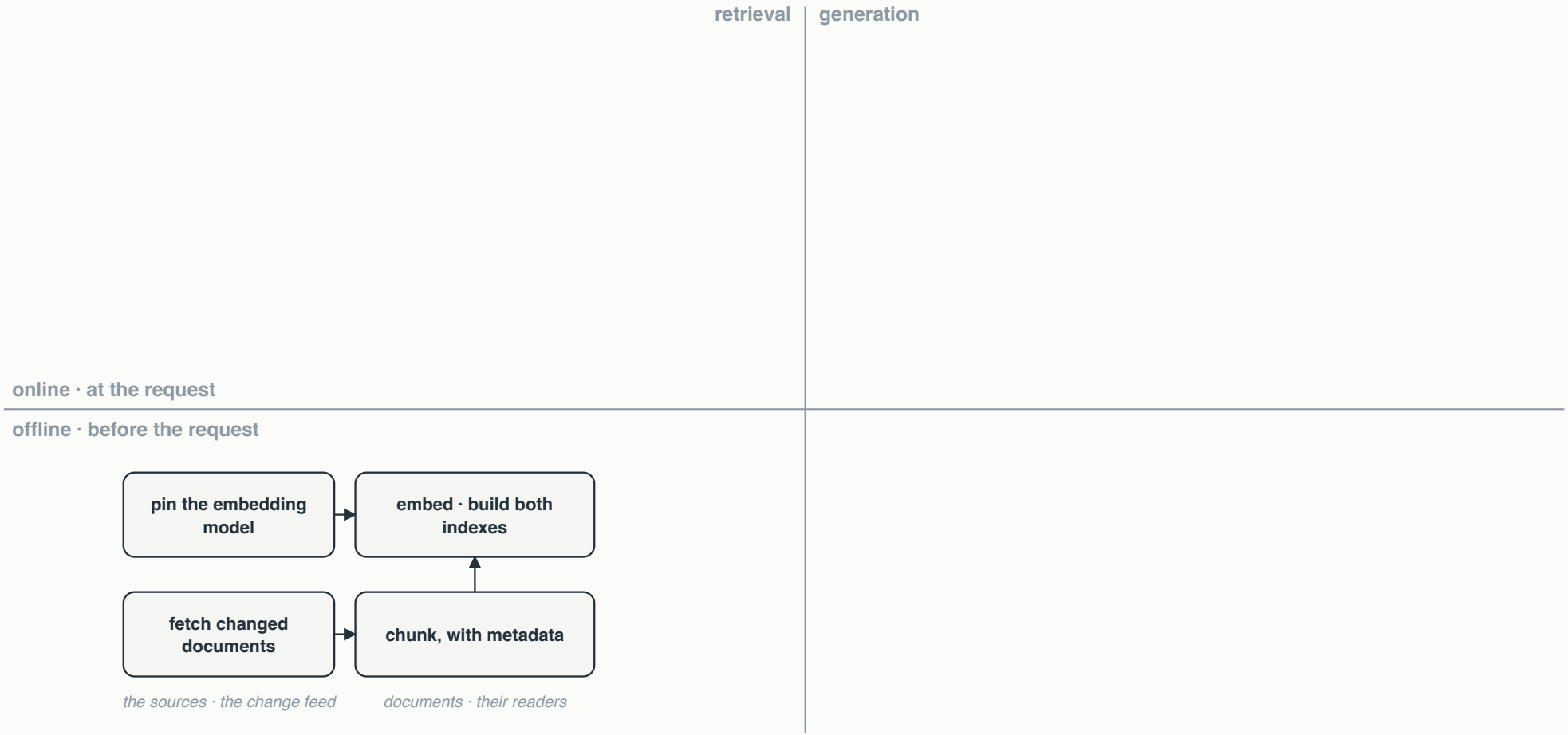
Everything about the documents is computed ahead; everything about this question is computed now; the index is the seam.

The retrieval-grounded assistant



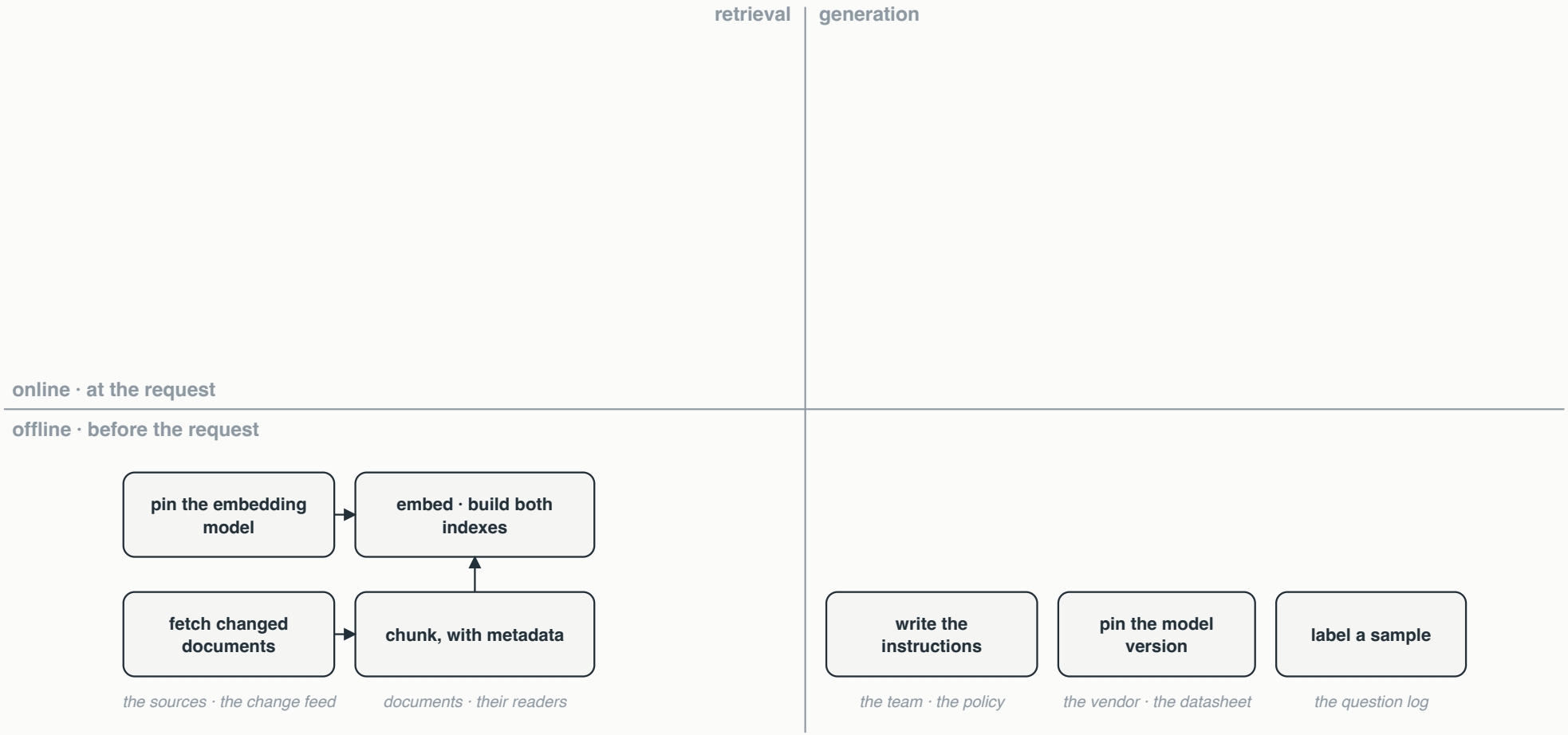
Above the line: what runs while the employee waits. Below: what ran before. Left: finding the passages. Right: writing the answer.

The retrieval-grounded assistant: offline, the retrieval side



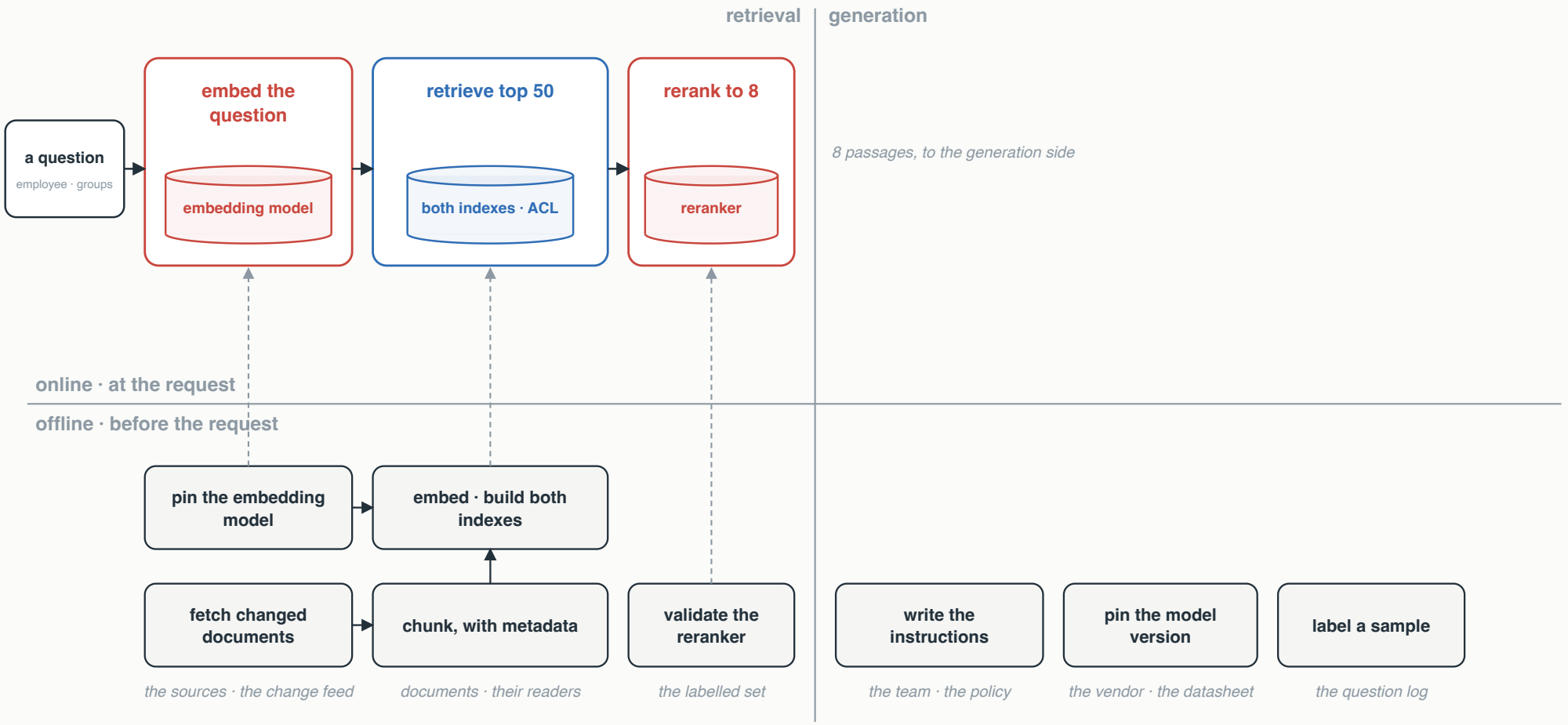
Offline, the retrieval side: changed documents are fetched and chunked, every chunk is embedded, both indexes are built.

The retrieval-grounded assistant: offline, the generation side



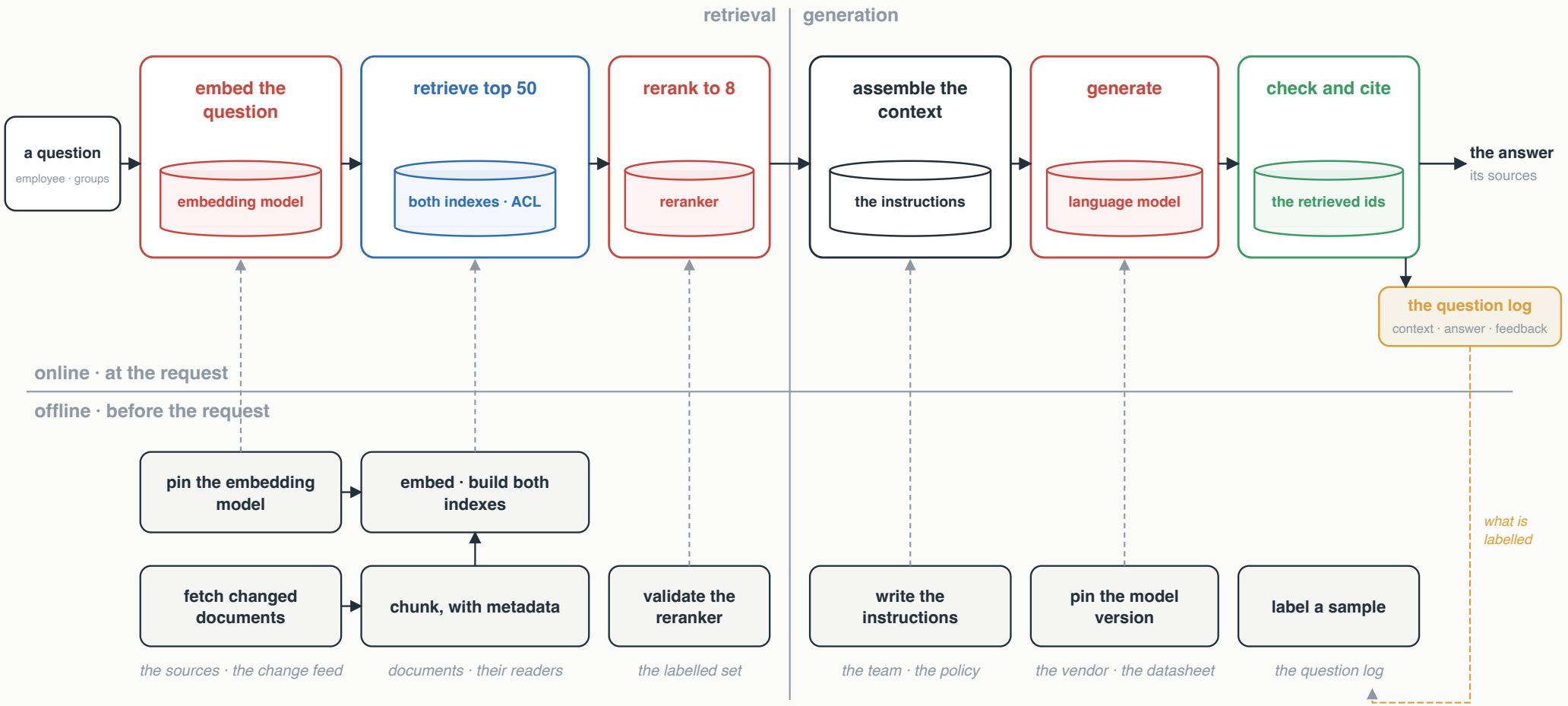
Offline, the generation side: the instructions are written by people, the model version is pinned, a sample of questions is labelled.

The retrieval-grounded assistant: online, retrieval



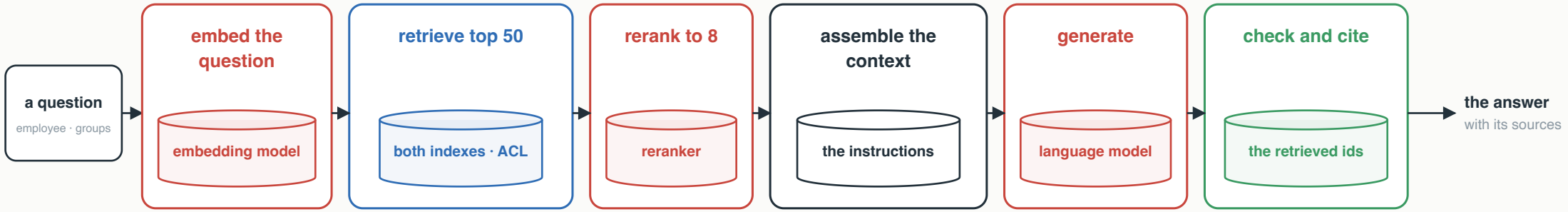
Online, the retrieval side: the question is embedded, both indexes return candidates the employee may read, the reranker keeps eight.

The retrieval-grounded assistant: online, generation



The whole system. Every online stage reads an artifact the offline side built; the question log is what the offline side labels.

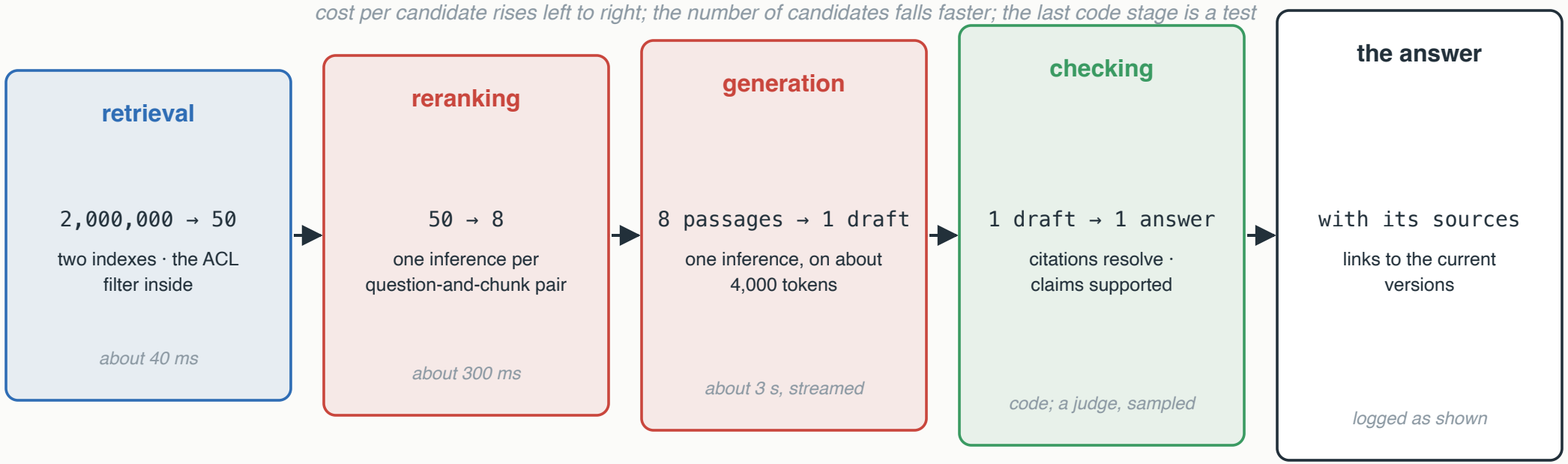
The request path



the request path: everything the employee waits for, left to right, inside 5 seconds

The top row of the diagram, on its own. Each stage in turn, left to right, with the artifact it reads and where that artifact came from.

The cascade for an assistant

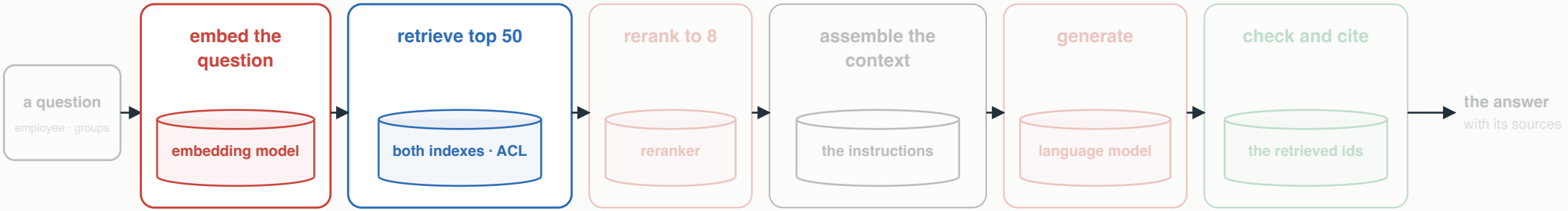


The generator sees only what retrieval returned and the reranker kept. A passage the indexes never return cannot be cited. Counts and times illustrative.

$$\text{cost per question} = N \cdot c_{\text{retrieval}} + k \cdot c_{\text{rerank}} + c_{\text{generate}}(m), \qquad k \ll N,\ m \ll k$$

Blue: retrieval, all N = 2,000,000 chunks through the indexes, a tiny cost each · red: one inference per candidate for the k = 50 reranked, then one generation over the m = 8 passages kept · the check is code and is left out.

The request path: find the candidates



the request path: everything the employee waits for, left to right, inside 5 seconds

What is the unit the index returns, and how is it found among 2,000,000?

The problem: a forty-page policy, a question about one sentence

The question. *"Can I expense a monitor for my home office? I'm in Berlin."*

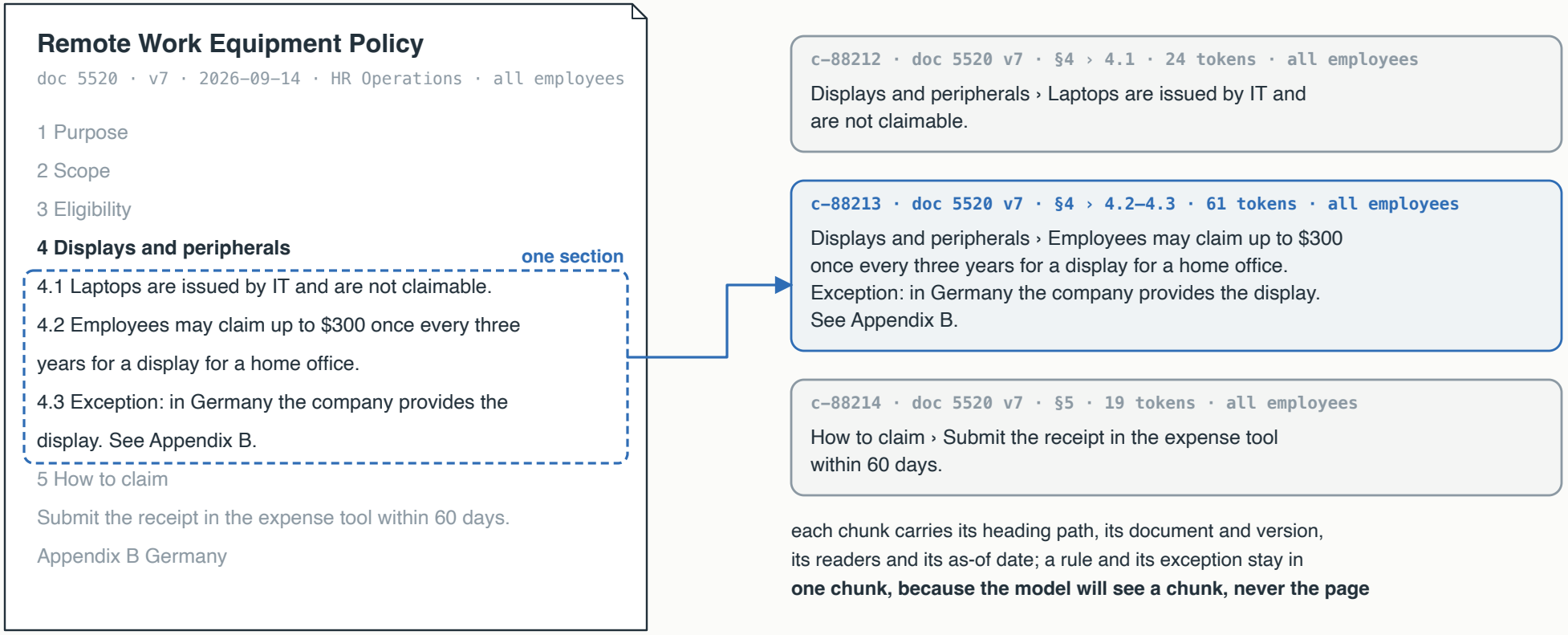
The answer is one sentence in section 4.2 of the *Remote Work Equipment Policy*, 40 pages, plus an exception in 4.3 and a procedure in section 5.

The budget. The model can be handed about 8 passages. The policy alone is 25,000 tokens.

IF THE UNIT IS	THEN
the document	eight documents fill the context with 200,000 tokens; the sentence is buried
the sentence	4.2 arrives without 4.3; the answer is wrong for Berlin
a fixed window of words	the window ends wherever it ends: mid-rule, mid-table

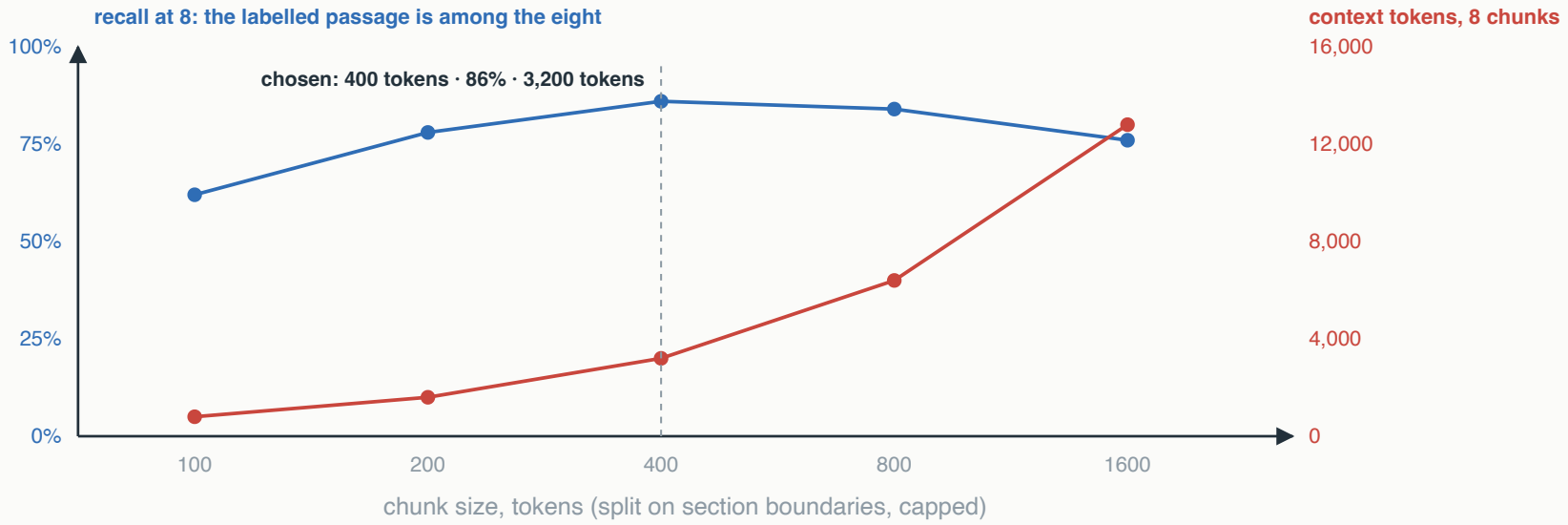
What should the index hold?

Resolution: chunk on the document's own structure, and carry the metadata



Chunking: a document becomes passages the index returns and the model reads. The unit of retrieval is the unit the model sees.

The dial: chunk size



Small chunks cut the answer from its condition; large chunks bury it and cost tokens. Measured on the labelled set; illustrative values.

A chunk, as a row

THE CHUNK CARRIES	WHY
its text, with the heading path prepended	what the index matches and the model reads
document id and version	the citation; and the old version's chunks are deleted by version
readers: the groups who may see it	the filter inside the search
as-of date	freshness, and "this answer was written from the policy as of…"
embedding model version	a vector from another model is a wrong answer that looks right
its position: section path, offset	the link lands on the passage, not the page

> Find every chunk that contains "ERR-4032", among 2,000,000, in a few milliseconds.

The lexical index

the chunks (four of 2,000,000)

c-104
claim up to $300 for a display once every three years

c-211
the home office monitor stipend was replaced by the display rule

c-377
VPN error ERR-4032 after the client update: reinstall the client

c-902
home office ergonomics: chair, desk and display height

tokenize

→

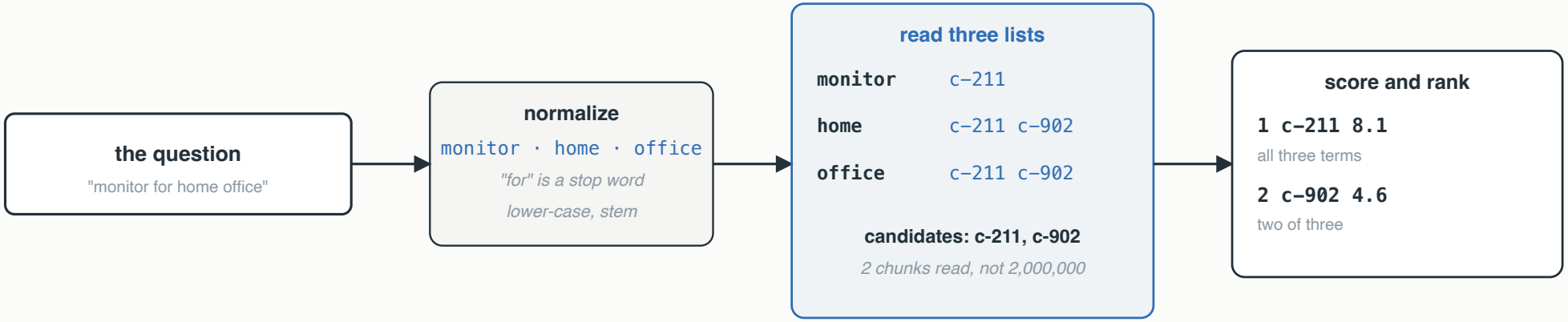
index

the inverted index: one list per term

TERM	DF	POSTINGS chunk : term count
claim	1	c-104:1
display	3	c-104:1 c-211:1 c-902:1
err-4032	1	c-377:1
home	2	c-211:1 c-902:1
monitor	1	c-211:1
office	2	c-211:1 c-902:1
stipend	1	c-211:1
update	1	c-377:1
vpn	1	c-377:1

The inverted index: for every term, the chunks that contain it and how often. Built offline over the whole corpus; a query reads a few lists.

Searching it



not on the list: c-104, the chunk with the $300 rule. It says "display"; the question said "monitor".

Lexical search: the query's terms select the posting lists; the chunks on them are scored by how rare and how repeated the matching terms are.

Scoring a chunk against a question

$$\text{score}(q, c) = \sum_{t \in q} \log\left(\frac{N - n_t + \frac{1}{2}}{n_t + \frac{1}{2}} + 1\right) \cdot \frac{f_{t,c}\,(k_1 + 1)}{f_{t,c} + k_1\left(1 - b + b\,\frac{|c|}{\bar{\ell}}\right)}$$

THE TERM	WHAT IT DOES
the first factor	a term in 3 chunks of 2,000,000 is worth far more than a term in half of them; "err-4032" outweighs "update"
the second factor	a chunk that says "display" three times beats one that says it once, but not three times as much
the length	a long chunk mentions everything; it is discounted toward the average

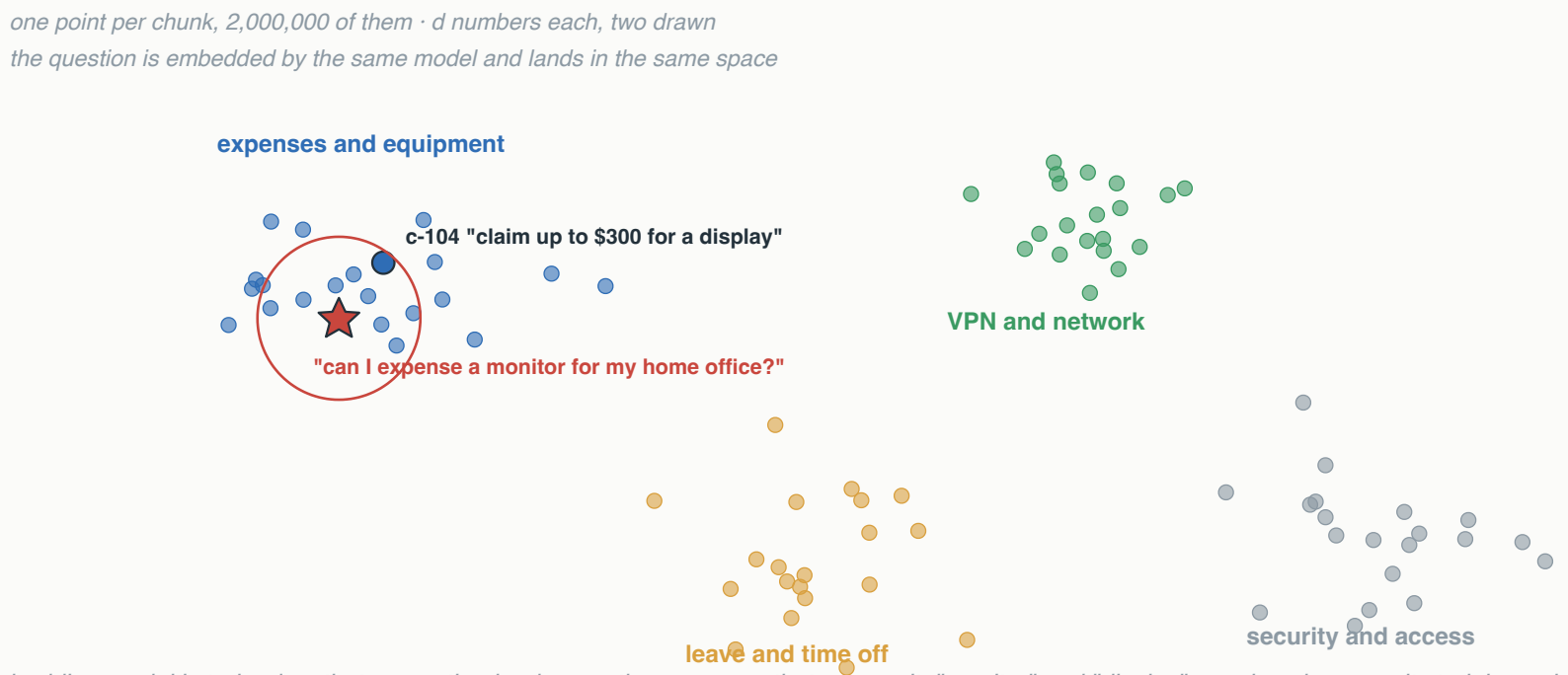
q: the question's terms · c: a chunk · N: chunks in the index · n_t: chunks containing term t · $f_{t,c}$: how often t occurs in c · $|c|$: the chunk's length in terms · $\bar{\ell}$: the average chunk length · $k_1 \approx 1.2$, $b \approx 0.75$: constants, rarely tuned. This is BM25 (1994); the reading's tf–idf is its first two factors without the saturation.

What words can find, and what they cannot

	the lexical index	the vector index
"Can I expense a monitor for my home office?"	✗ **c-211 found: "monitor stipend"** *c-104 missed: it says "display"*	✓ **c-104 found: the $300 rule** *c-211 found: the stipend note*
"VPN fails with ERR-4032 after the update"	✓ **c-377 found: the exact code** *one list, one chunk, 2 ms*	✗ **the general VPN pages come first** *c-377 ranked 40th: the code is one token among many*

Each index misses what the other finds: the lexical index is exact and literal; the vector index is semantic and blurry.

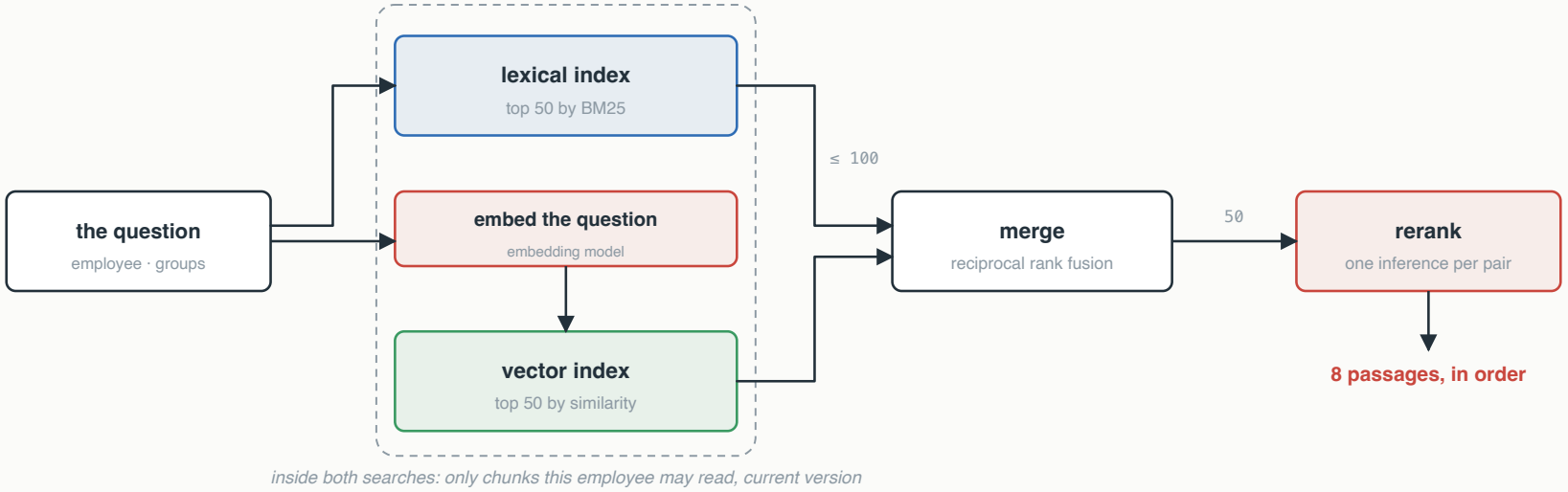
The vector index, for text



A text embedding model is trained so that a question lands near the passages that answer it; "monitor" and "display" are close because its training pairs said so.

The same component as a feed's video vectors: an embedding model, a vector per chunk computed once offline, an approximate nearest-neighbour index with its cells-probed dial, and a question embedded at the request by the same model.

Resolution: both indexes, then a model that reads the pair

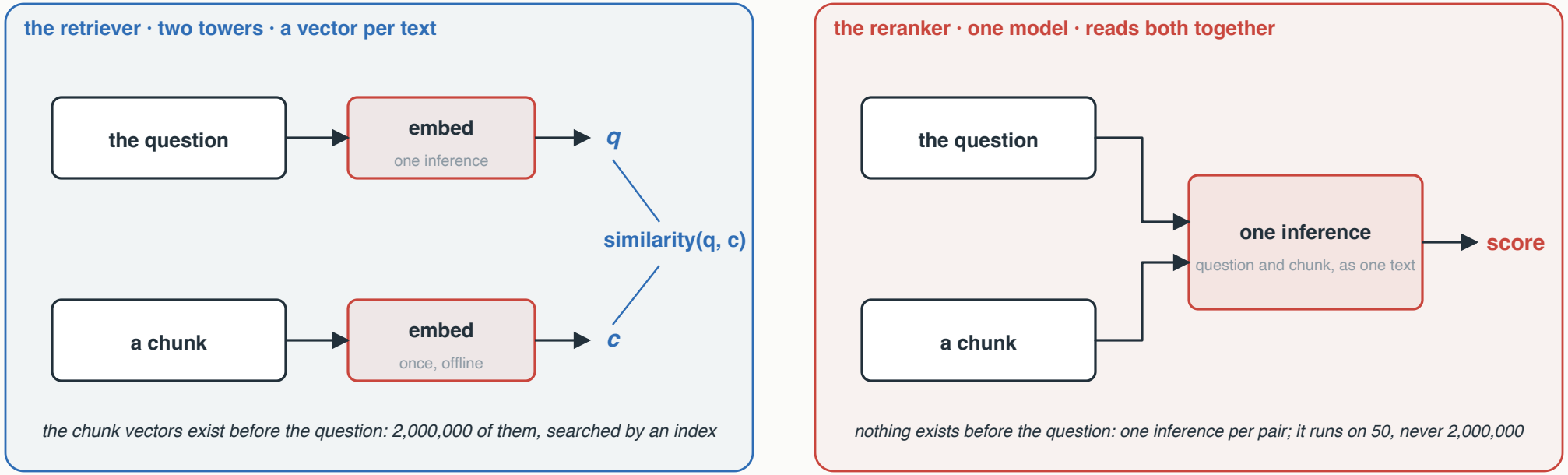


Hybrid retrieval: each index finds what the other cannot; the merged list is reranked by a model that reads the question and the chunk together.

$$\text{fused}(c) = \sum_{r \in \{\text{lexical, vector}\}} \frac{1}{60 + \text{rank}_r(c)}$$

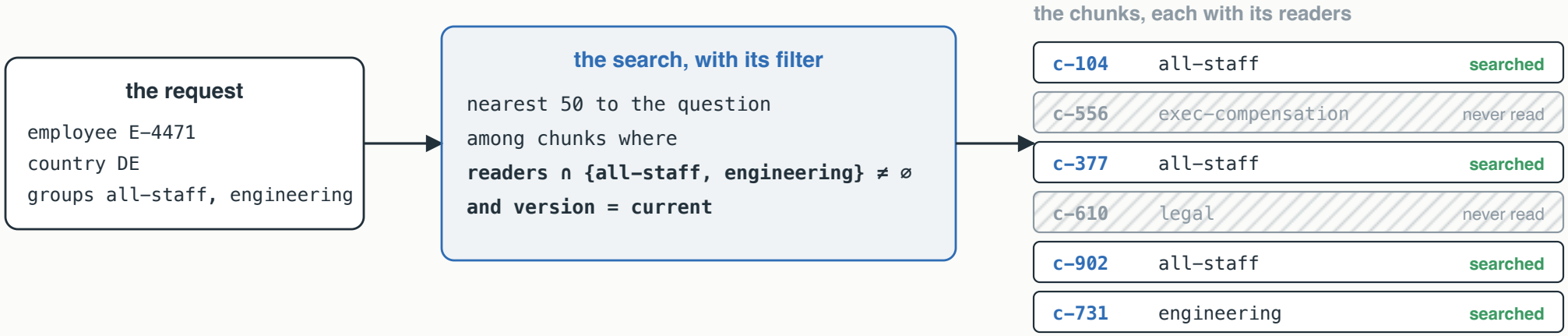
c: a chunk · $\text{rank}_r(c)$: its position in index r's list, or absent · 60: a constant that keeps the top ranks from dominating. Reciprocal rank fusion needs only the two orders, not comparable scores.

The reranker: one model reads the question and the chunk together



Two ways to score a question against a chunk: the towers can be searched by an index; the cross-encoder can see that "display" answers "monitor".

Filtering inside the search: who may read this

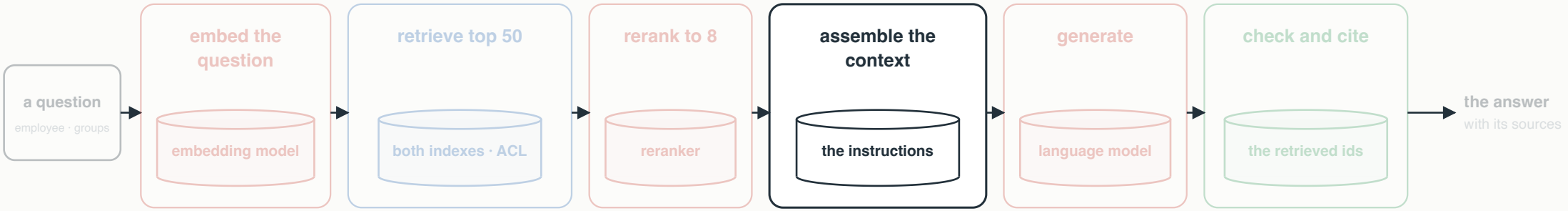


The filter runs inside the search, before any passage is read. A passage the model is given is a passage this employee may read.

The readers of a chunk are copied from the document's own permissions when it is indexed, and refreshed with the document.

Access control at retrieval time: the question carries who is asking, and the index answers only from what they may read.

The request path: assemble the context



the request path: everything the employee waits for, left to right, inside 5 seconds

Eight passages and a question. What exactly does the model receive?

What the model receives

SYSTEM INSTRUCTION

You answer employees' questions using only the passages below.
Every sentence that states a fact cites its passage as [n]. If the passages do not
answer the question, reply exactly NOT_FOUND. Do not use other knowledge.
Readers may not see the passages; quote the figure and the condition, not "see above".

PASSAGES · 8, in reranker order, each with its id and metadata

[1] c-88213 · Remote Work Equipment Policy v7 · §4 Displays › 4.2–4.3 · as of 2026-09-14
Employees may claim up to $300 once every three years for a display for a home office. Exception: in Germany…

[2] c-88214 · Remote Work Equipment Policy v7 · §5 How to claim · as of 2026-09-14
Submit the receipt in the expense tool within 60 days.

[3] c-211 · Home Office FAQ v12 · Stipend · as of 2026-03-02
The home office monitor stipend was replaced by the display rule in the equipment policy.

[4] … [8]

QUESTION

Can I expense a monitor for my home office? I'm in Berlin.

tokens

instruction	≈ 150
8 passages	≈ 3,200
question	≈ 20
in all	**≈ 3,400**

THE OUTPUT CONTRACT

an answer of a few sentences,
each fact followed by [n]
with n among 1…8;
or the single token
NOT_FOUND

everything the model knows at this request is in the box on the left; nothing else.

rendered by code from a template, logged whole.

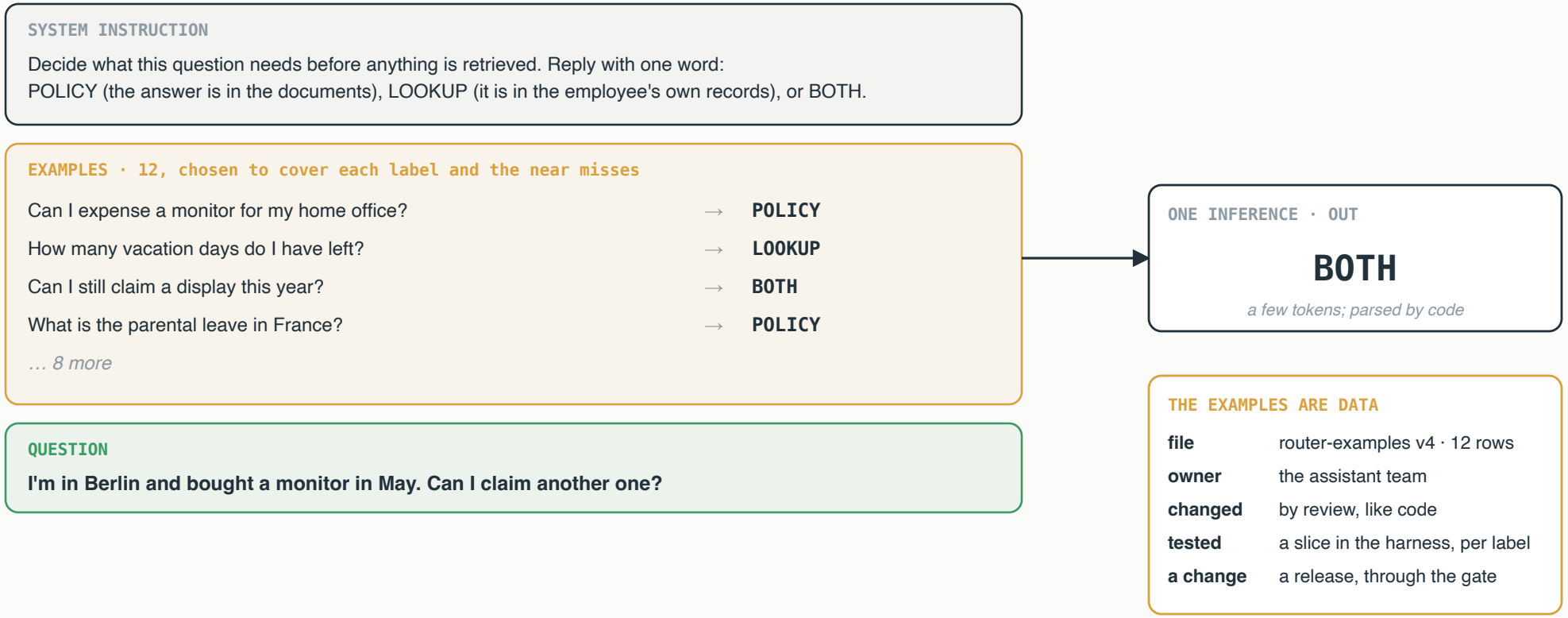
What the model receives: the instructions, the eight passages with their ids, the question. The context is the model's entire world for this answer.

The instructions, as a contract

THE INSTRUCTION SAYS	WHY IT IS THERE
answer only from the passages	the model's own knowledge is a different company's policy, or an old one
cite every fact as [n]	the check needs something to check, and the employee needs somewhere to go
reply NOT_FOUND when the passages do not answer	the alternative is a fluent guess; the not-found path is designed, not accidental
quote the figure and the condition	"$300 every three years, except in Germany" survives the reader not opening the source
a few sentences	the budget, and the checker's limit

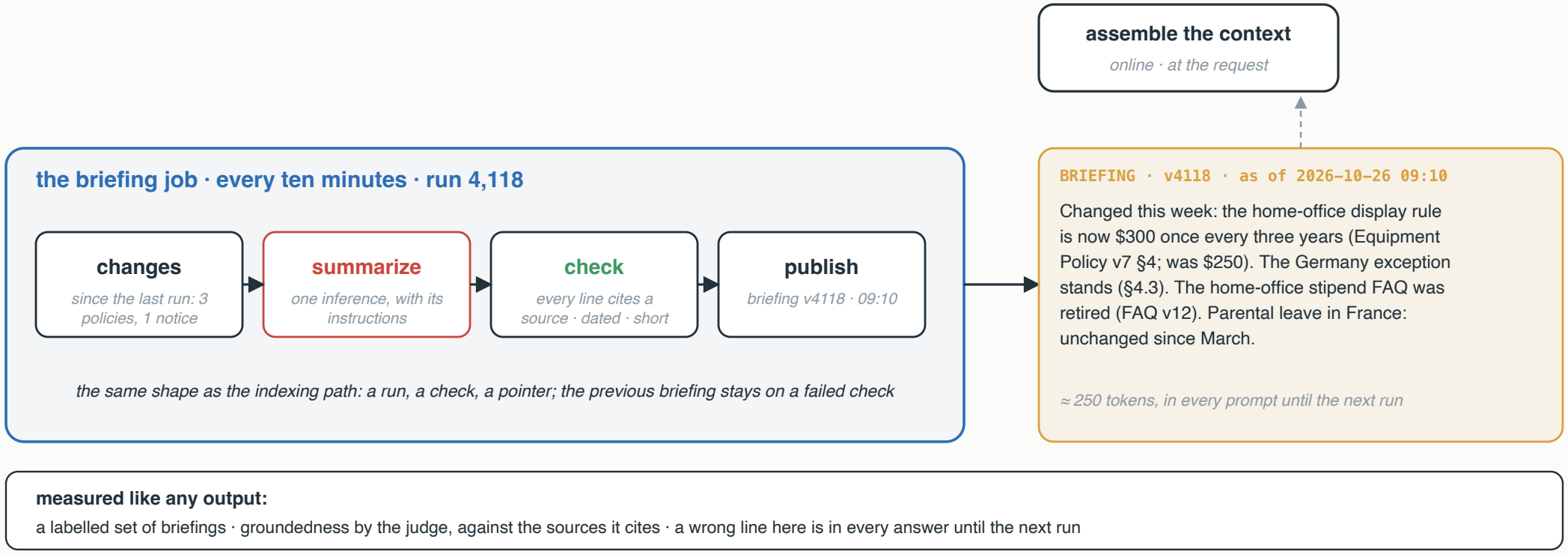
The instructions are a requirement written in prose and versioned like code. Every line is tested by the harness.

Examples in the prompt



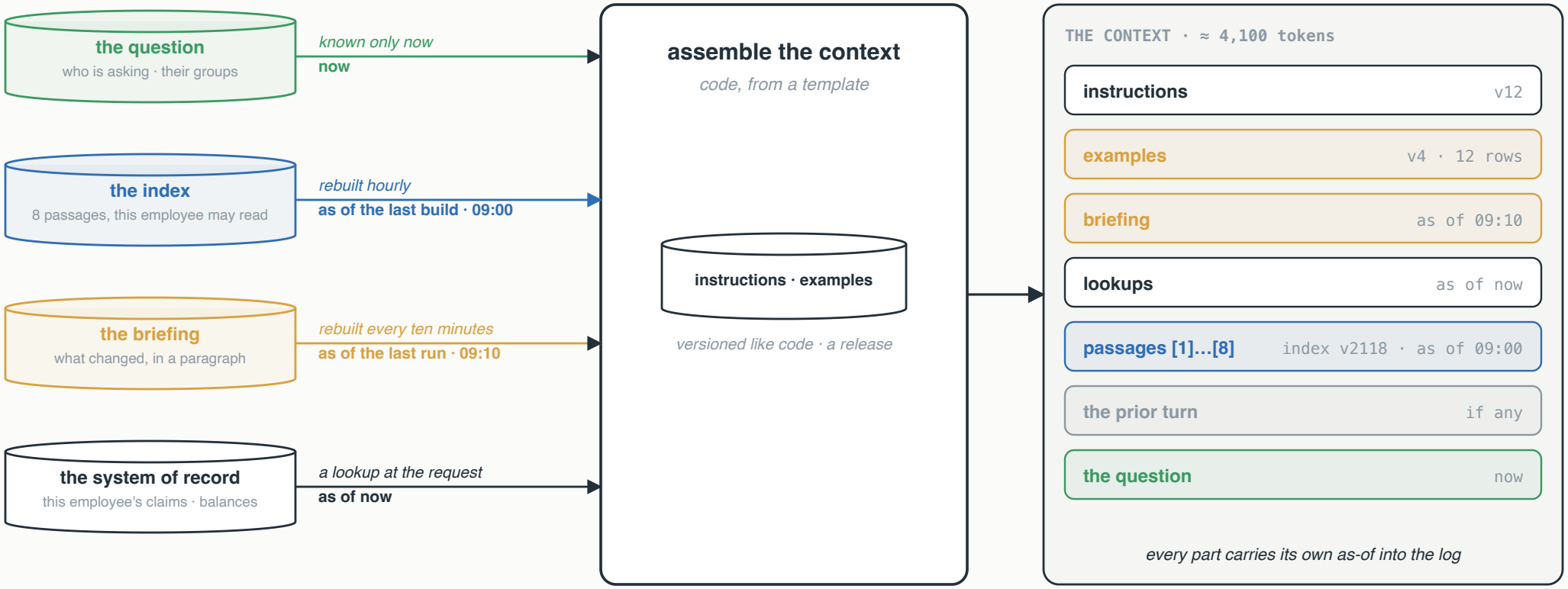
The router: a few worked examples in the prompt teach the model the labels. No training; the examples are a dataset with a version and a test.

A summary written before the request



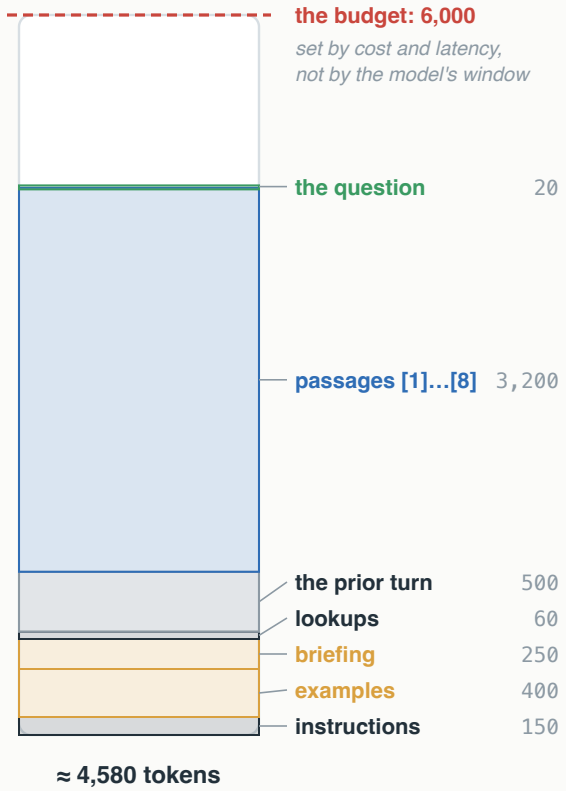
A summary the model writes before any question arrives, built every few minutes, read at the request like the instructions and the index.

Where the context comes from



Where the context comes from: the index, rebuilt hourly; the briefing, every few minutes; a lookup, now. Each part is as old as its source.

The context has a budget



when it does not fit, in this order

1. **passages**
   8 → 6: the reranker's tail
2. **the prior turn**
   summarized to a line by one inference
3. **examples**
   12 → 6: the ones the harness says it can spare

the briefing
never dropped; capped at the job

instructions · question
never

THE SECOND QUESTION

turn 1
Can I expense a monitor? I'm in Berlin.
Yes: up to $300 every three years [1]; in Germany … [3]

turn 2
And if I already claimed one in May?

↓

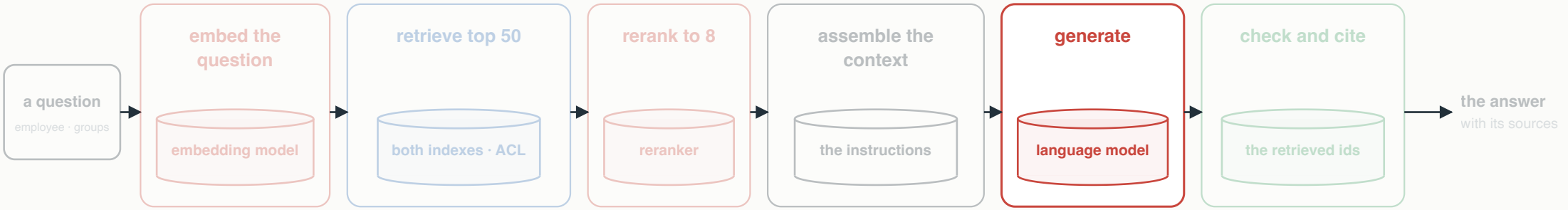
rewrite, one inference, from the prior turn:
`"a second display claim within three years, Germany"`

→ retrieve again, with the rewritten question

carried	the prior question and answer, the ids it cited
not carried	the prior passages: retrieved again, fresh
kept where	the conversation table, one row per turn
logged	the whole context, both turns, as given

The context has a budget, and what gives way is decided in code, not by the model. A follow-up carries the turn, not the passages.

The request path: generate



the request path: everything the employee waits for, left to right, inside 5 seconds

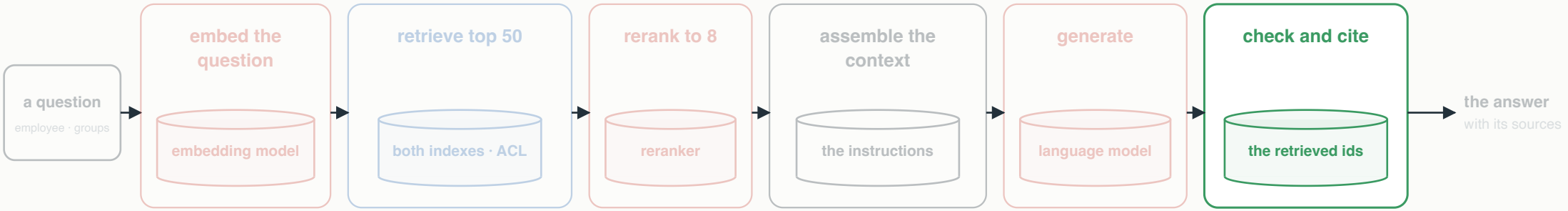
One inference, on about 4,000 tokens. What comes back, and what do we know about the component that produced it?

The language model as a component

THE MODEL	
input	the context: instructions, passages, question; about 3,400 tokens
output	text: a draft of a few sentences with [n] citations, or NOT_FOUND
the contract	the output format is asked for in the instructions and **enforced by the check**, never assumed

PROPERTIES	
latency	2 to 4 seconds for 120 tokens of output; streamed, so the first word arrives in under a second
cost	priced per input and per output token; the eight passages are most of it
nondeterminism	the same context can produce a different draft; the check runs on every one
version	the vendor's, pinned in the pointer; a vendor's update is a release of ours, through the same gate

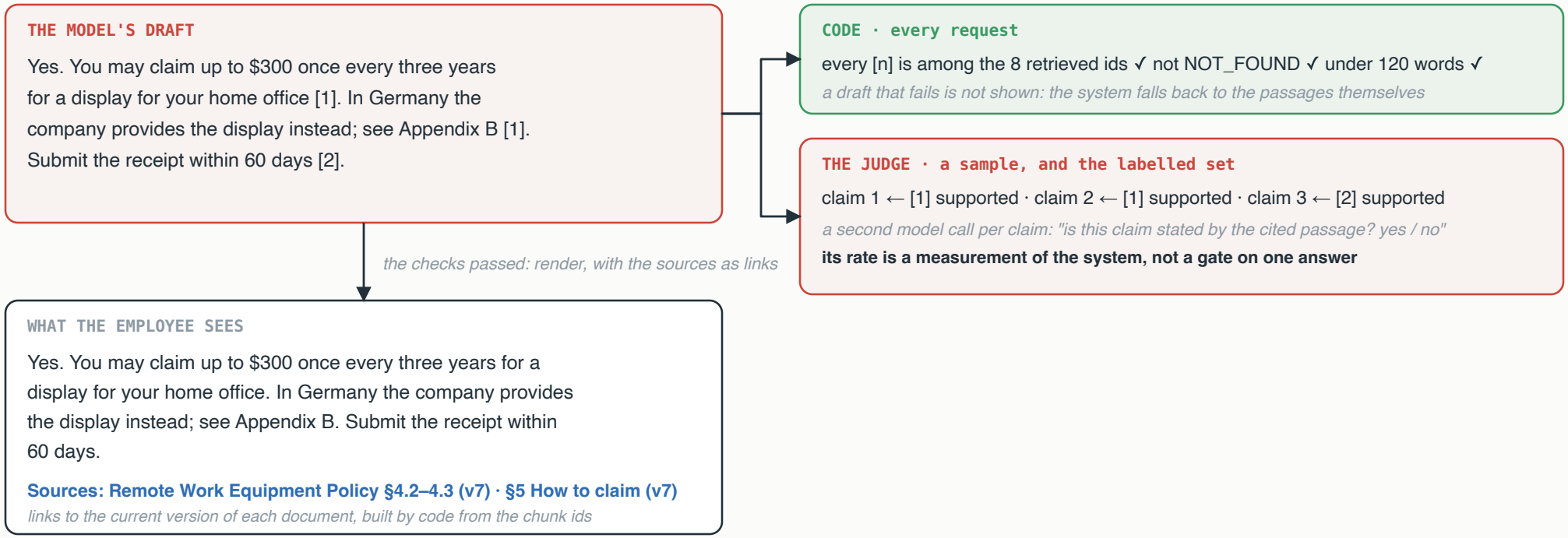
The request path: check and cite



the request path: everything the employee waits for, left to right, inside 5 seconds

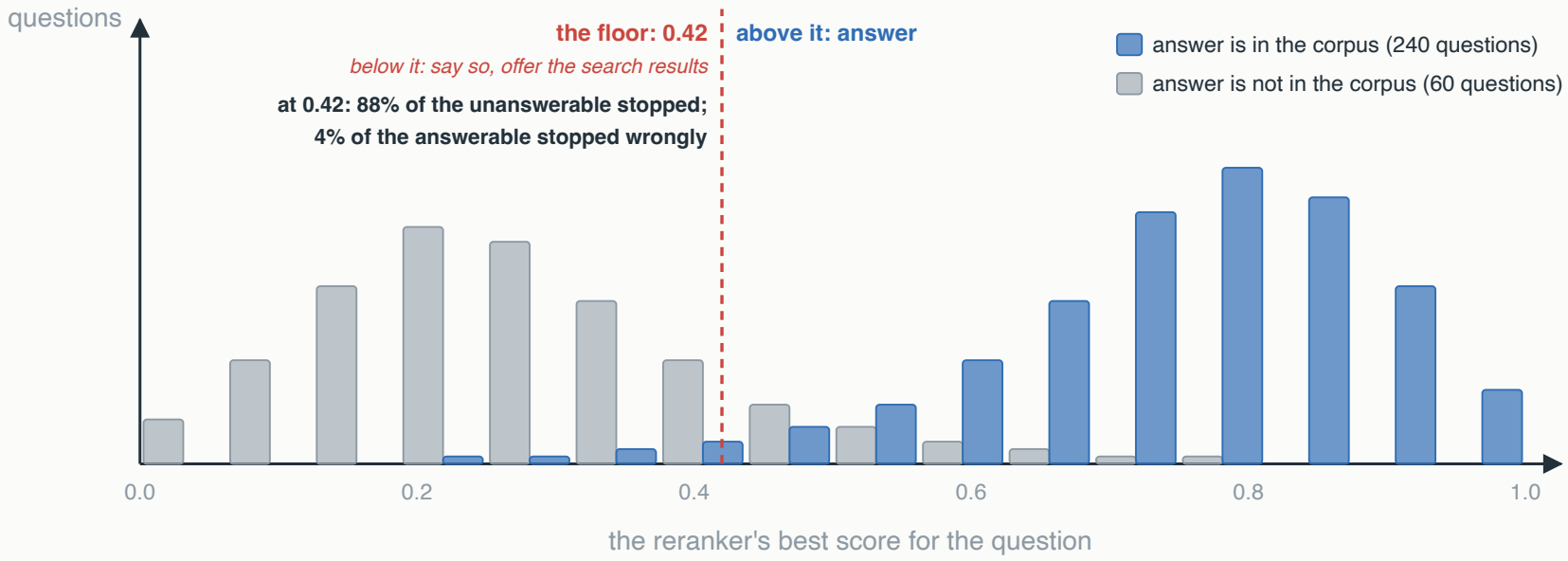
The draft has three citations. What does code check, what does a judge check, and what does the employee see?

Checking the draft



Citing is the model's job. Checking is code on every answer and a judge on a sample. The links are built by the system, not written by the model.

When nothing answers the question



The not-found decision is a threshold on a score, measured on the labelled set with its unanswerable slice.

What is computed before the request, and what at it

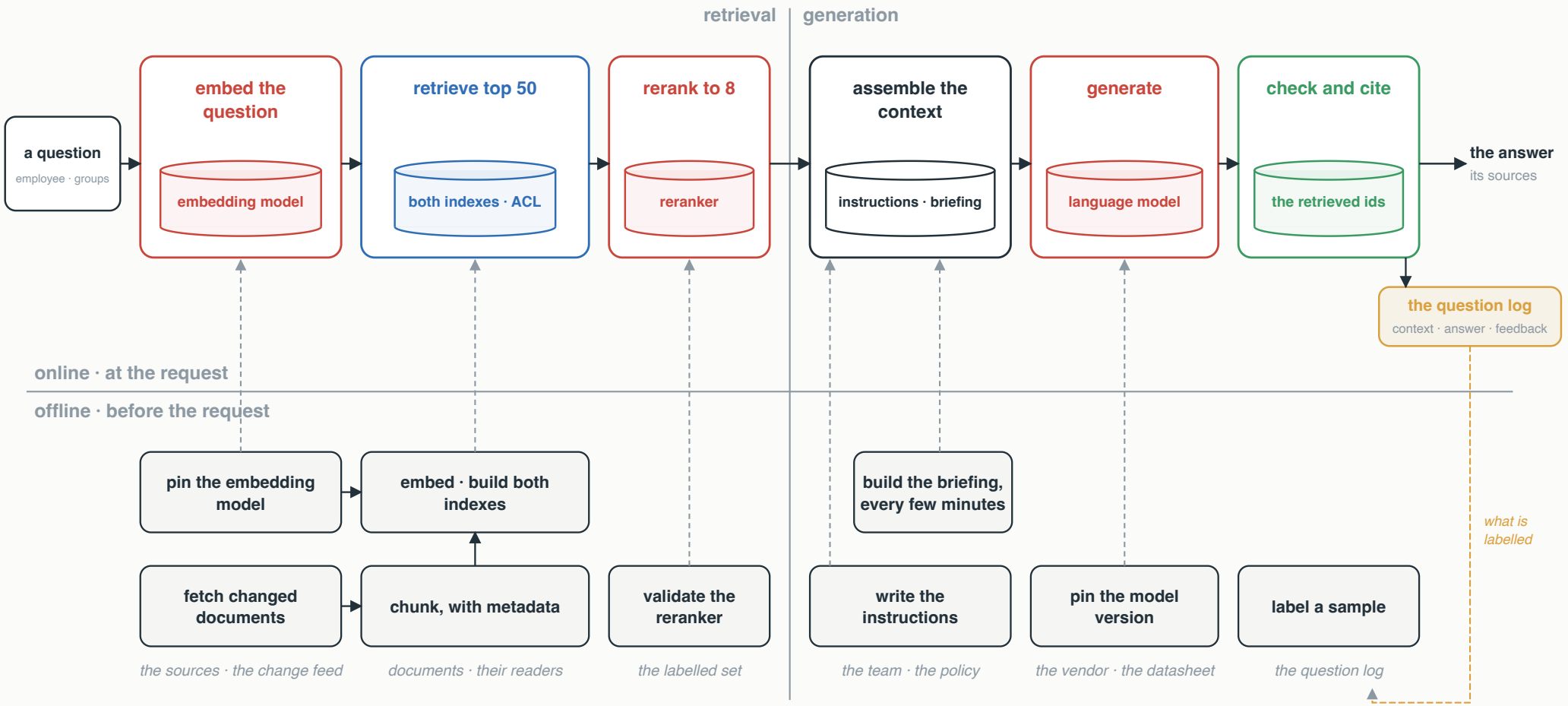
COMPUTED	BEFORE THE REQUEST	AT THE REQUEST
about every document	the chunks · both indexes · the readers · the as-of	
about the question		its embedding · its terms · who is asking
about the pair		the candidates · the fused order · the reranker's eight
the answer	the instructions · the examples · the model version	the context · the draft · the checks · the links
the briefing	every ten minutes: one inference, a check, a pointer	read, with its as-of
the lookups		this employee's records, by key, from the system of record
the seam	the indexes, the chunk table and the briefing, each with a version and an as-of	read, never written
the dial	chunk size · how often the index is rebuilt · how often the briefing runs	candidates fused · passages kept · the floor · the budget

As of 2020: retrieval-augmented generation

THE READING (LEWIS ET AL.)	ON THE DIAGRAM
a **dense retriever** over 21,000,000 passages of 100 words from Wikipedia, with an approximate nearest-neighbour index; the query encoder and the passage encoder are two models	the **vector index** over the chunks, with the **embedding model** on both sides; the unit of retrieval is a fixed 100 words
a **generator** that reads the question with the top *k* passages and writes the answer; the paper's *k* is 5 to 10	the **language model** given the **context**; our *k* is 8
retriever and generator **trained together** so that the retriever learns what the generator needs	**not done in production**: the embedding model and the language model are the vendors', pinned; what we tune is the chunking, the fusion, the reranker, the instructions
the retrieved passages are **marginalized**: the answer is averaged over which passage was used	**dropped**: a production answer cites the passages it used, and the check needs them named

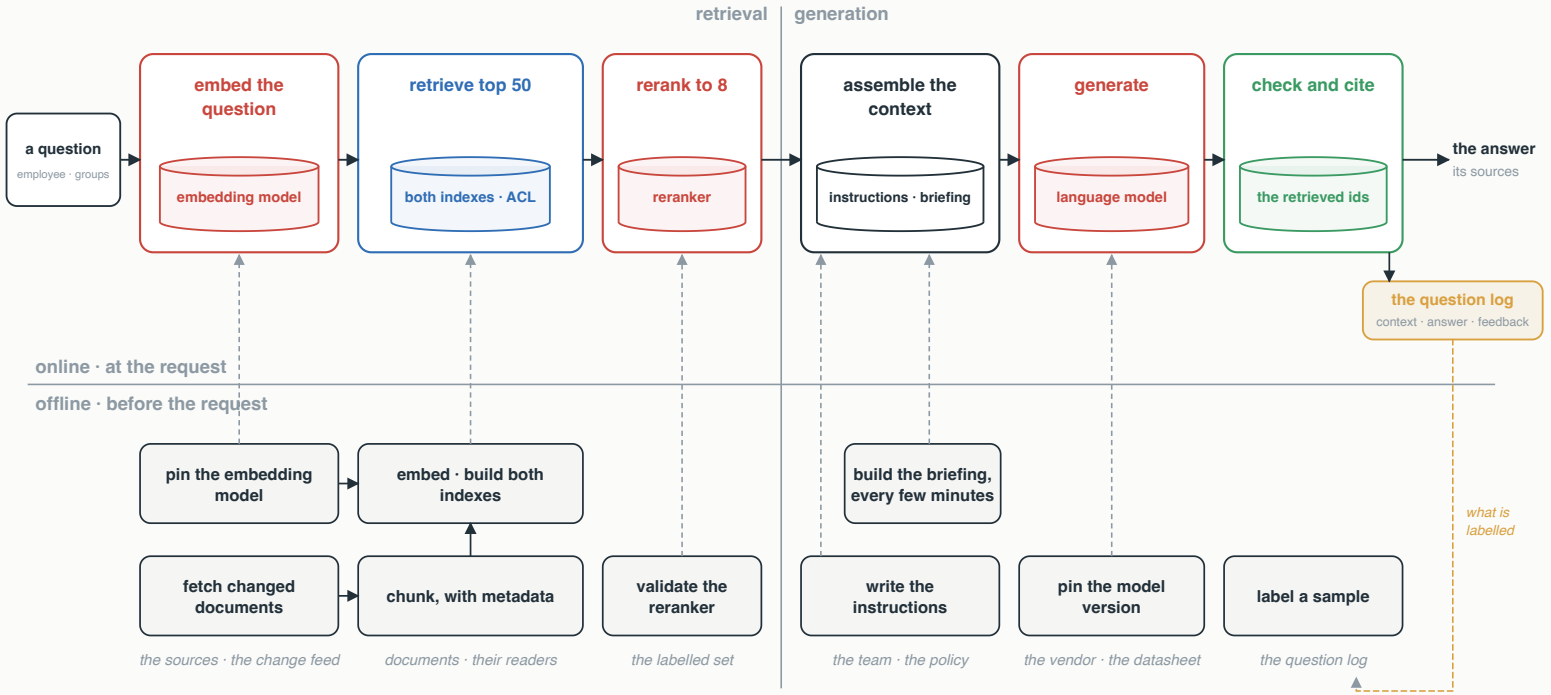
Lewis et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," NeurIPS 2020. The name stuck; the architecture that stuck is retrieve-then-generate, with the retriever and the generator as separate, replaceable components.

The retrieval-grounded assistant, complete



The whole system, with the briefing: a summary built every few minutes, read by the assemble stage as the instructions are.

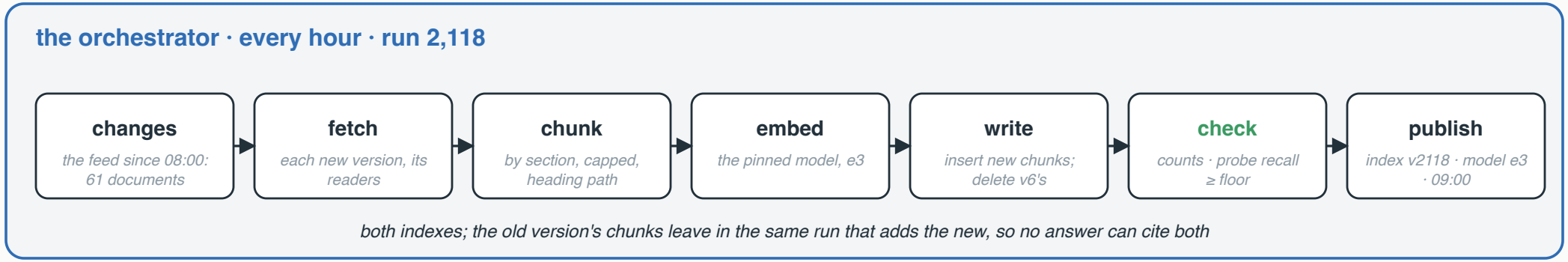
The retrieval-grounded assistant, complete



The whole system, with the briefing: a summary built every few minutes, read by the assemble stage as the instructions are.

The display policy changed at 8:40 this morning. What has to happen before the assistant's answer changes?

The indexing path



the pointer the request path reads:

```
index v2118 · embedding model e3 · as of 2026-10-26 09:00 · 2,004,311 chunks · probe recall 0.91 · on a failed check, v2117 stays
```

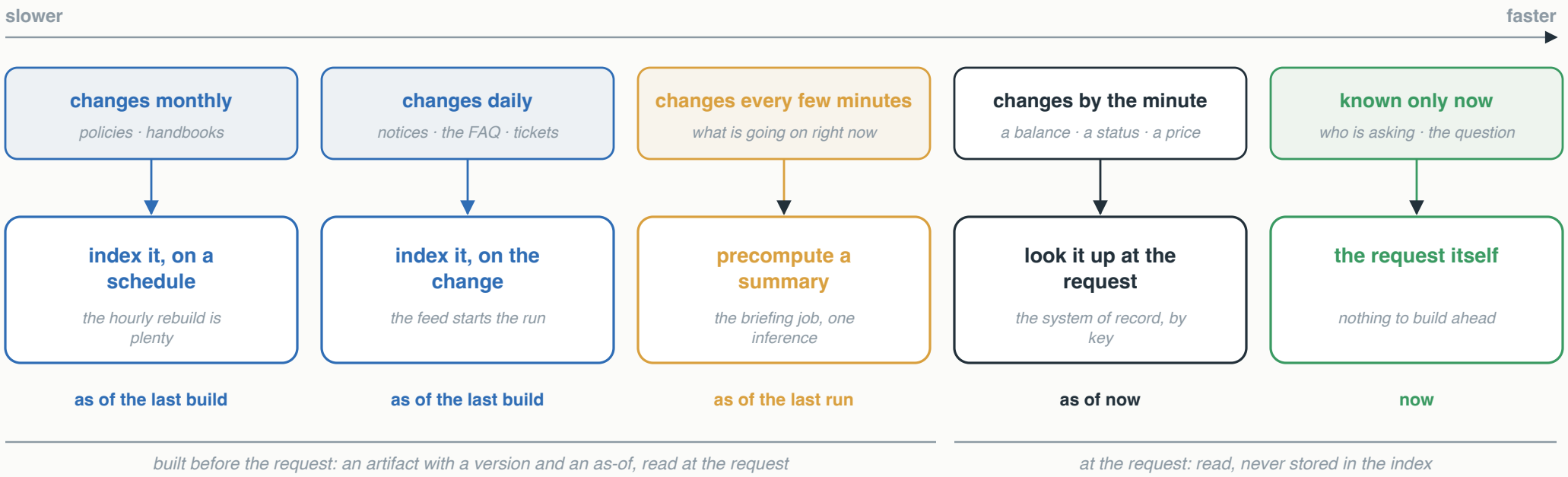
The indexing path: a batch job like any other, with a check and a pointer. A document changed at 08:40 is searchable, and its old version is not, by 09:05.

One corpus, several kinds of document

KIND	CHANGES	CHUNKED	EMBEDDED	CARRIED AS METADATA
a policy	monthly	by clause, with the heading path, capped	the clause, title and path prepended	version · as-of · readers · section
a notice	daily	one chunk: short, and the condition travels with the rule	the whole notice	date · who it applies to · expiry
a FAQ	daily	one chunk per question and answer	the question and the answer together	the document it points at
a table	weekly	one chunk per row, the header repeated	a sentence rendered from the row	the row key
a ticket thread	by the hour	the thread, re-chunked on every reply	the thread so far	status · last reply · the asker

One index and one embedding model. The chunking rule is per kind, and so is the as-of.

The freshness ladder



What changes faster than the index is rebuilt is never in the index.

One mechanism per rate of change. The dial on each rung is how stale an answer may be, and every rung writes its own as-of into the log.

Handling failures in a retrieval-grounded assistant

When the assistant fails

WHEN	WHAT RUNS	WHAT THE EMPLOYEE SEES
the index build fails its check	the **previous index** stays in service; the pointer did not move	answers from the corpus as of an hour ago
the vector index is down	the **lexical index alone**; recall is lower and measured	an answer, more often "not found"
the reranker exceeds its budget	the **fused order**: the indexes' ranks are an order too	a worse set of passages, on time
nothing clears the floor	**no generation**; the search results and a way to ask a person	"I could not find this; here is what matched"
the draft fails the check	**not shown**; the passages themselves, with links	the sources without the paragraph
the language model is down or slow	**the passages themselves**, with links	search results, as it was before the assistant

Everything here degrades the answer to a search result. Nothing degrades it to a guess.

The data

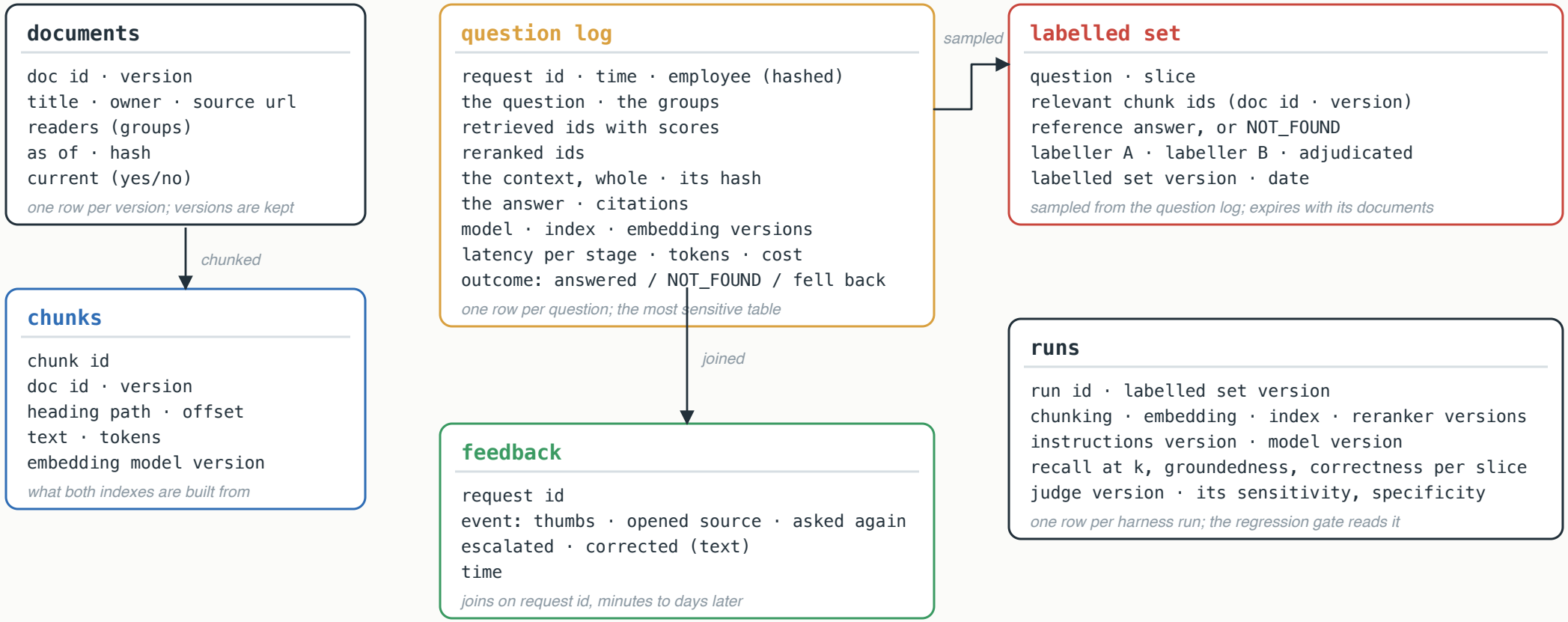
Where do the documents come from, and who may read them?

The corpus as a dataset

THE CORPUS HAS	WHICH MEANS
sources: the wiki, the policy repository, the IT knowledge base, resolved tickets	each with its own change feed, owner and permission model; a connector per source
provenance: owner, source URL, hash, as-of, per version	a citation points at a version; a dispute is settled by reading it
permissions, copied from the source at indexing time and refreshed with it	the readers column on every chunk; a permission change is an index change
freshness: change to searchable within the hour	the age of the newest version the index can return, measured
invalidation: the old version's chunks leave when the new one arrives	no answer cites a version that is no longer current
quality: a wrong page is a wrong answer, with a citation	the corpus has owners; a thumbs-down routes to the page's owner, not to the model's

A document in the corpus is input to the model. Whoever can edit a page can write to the prompt.

The tables of a retrieval-grounded assistant



The tables of a retrieval-grounded assistant. The question log holds what the model saw; the labelled set says what it should have found and said.

The context is data the model was given

LOG, WHOLE, PER REQUEST	WHY
the context: instructions version, the eight passages by id, the question	what the model saw is the only explanation of what it said
the retrieved fifty with their scores, and the fused order	to tell a retrieval failure from a generation failure, after the fact
the draft and the final answer, with citations	the check's verdict, and the judge's, attach here
every version: chunking, embedding model, index, reranker, instructions, model	to reproduce the answer; to attribute a change in a rate to a change in a version
retention and access	restricted passages are in this table; it inherits the strictest permission among them

What does the system know about whether an answer was right?

What arrives on its own, and what it means

THE EVENT	IT SAYS	IT DOES NOT SAY
thumbs down	this employee was unhappy with this answer	whether the answer was wrong, or unwelcome, or right but about the wrong thing
opened a source	the employee wanted to see the document	whether it confirmed or contradicted the answer
asked again, reworded	the first answer did not do	which stage failed
escalated to a person	the assistant did not resolve it	whether it was wrong or merely not allowed to act
nothing, for 95% of answers		anything at all

No label arrives on its own. Every label for text is made by a person, against a written rule.

The problem: 20,000 answers a day, and the truth costs ten minutes each

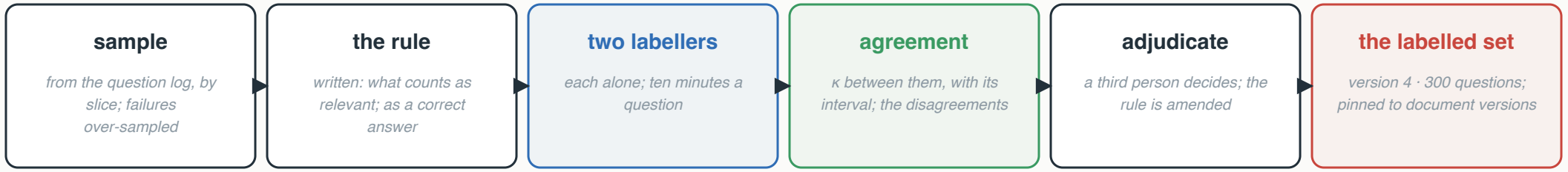
To know whether the system is right you need, for a sample of real questions: which chunks answer it, and what a correct answer says.

A person with the corpus open needs about **ten minutes** per question to find the passages and write the reference. Two people, so that you know whether they agree.

THE ARITHMETIC	
sample	300 questions
people	2, each alone
time	300 × 2 × 10 min ≈ **100 hours**
plus	adjudication of the disagreements
and again	when the documents those labels point at change

How is a label for text made, and how do you know it is any good?

Resolution: labelling as a pipeline



where a rule is unclear, the two disagree: the disagreement is the requirement, not yet written

label again when:	a document a label points at has a new version · the rule changes · a new slice appears in the log · on a schedule, a fresh sample
cost:	300 questions × 2 labellers × 10 minutes ≈ 100 hours, plus adjudication; a label for text is bought, never collected

Labelling as a pipeline: the label is made by people against a written rule, measured for agreement, versioned, and remade when its documents change.

The rule the labellers follow

THE QUESTION	THE WRITTEN RULE
which chunks are relevant?	a chunk is relevant if a correct answer could be written from it alone, or from it with one other chunk; list every such chunk
what is the reference answer?	the figure, the condition, the procedure, in one to three sentences, each with the chunk it comes from
when is the answer "not in the corpus"?	when no chunk, and no two chunks, answer it; the labeller records what they searched for
what slice is it on?	domain (HR · IT · expenses · engineering) · country · kind (fact · procedure · yes/no) · an identifier in the question · the document changed in the last 30 days

Where two labellers disagree, the rule is incomplete. The disagreement is written down and the rule is amended before the next batch.

Example: do the two labellers agree?

Two people label the same **50 claims** from 15 answers as supported or not by the cited passage:

	B: SUPPORTED	B: UNSUPPORTED
A: supported	38	3
A: unsupported	2	7

Raw agreement 45 of 50 = 90%

$$\kappa = \frac{p_o - p_e}{1 - p_e} = \frac{0.90 - 0.69}{1 - 0.69} = 0.68$$

95% interval **0.41 to 0.95**, from 50 items.

- most claims are supported, so two people saying "supported" agree partly **by chance**; κ removes that
- the **5 disagreements** are read, and the rule gains a line
- the humans' κ is **the ceiling**: no judge validated against these labels can be shown to do better than 0.68

p_o: observed agreement, 45/50 · p_e: agreement expected by chance from each labeller's own rates, (41/50)(40/50) + (9/50)(10/50) = 0.69 · interval from κ's standard error ≈ $\sqrt{(\, p_o(1-p_o) \,/\, (n(1-p_e)^2)\,)}$ ≈ 0.14.

The labelled set

THE LABELLED SET	
one row	the question · its slice · the relevant chunk ids, pinned to document versions · the reference answer, or NOT_FOUND · both labels and the adjudicated one
its slices	HR · IT · expenses · engineering · each country · fact / procedure / yes-no · identifiers in the question · changed in the last 30 days · not in the corpus (60 of 300)
its version	the rule's version, the labellers, the date; the harness carries the number
when it expires	a document a row points at has a new version: the row is re-labelled or retired; the harness reports how many rows are current
what it costs	about a hundred hours per three hundred questions; the budget for truth

The unanswerable slice is in the set on purpose. A system that is never asked what it cannot answer is never measured on saying so.

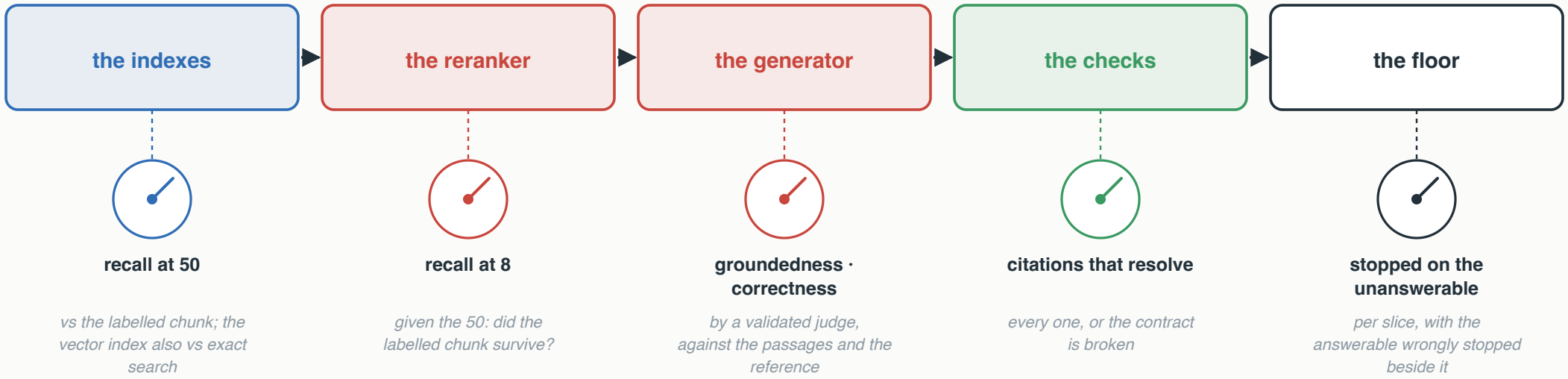
The measurement

What we measure, and why

WHAT	THE MEASURES	WHY
retrieval	recall at *k*, per retriever, per slice, against the labelled chunks; the floor's stop rate on the unanswerable	the generator cannot cite what was not retrieved; measured first, without any generated text
generation	groundedness: claims supported by the cited passages · correctness against the reference · the "not found" rate where it should and should not be	the answer is read and acted on; a fluent unsupported sentence is the failure of the week
the instrument	the judge's sensitivity, specificity and κ against adjudicated labels; its own repeat noise; the corrected rate	the groundedness rate is only as good as the judge that produced it
the release gate	helped · hurt · cannot tell, per slice, for every version change; guardrails on latency, cost and the stop rate	a change to chunking, a prompt, or the vendor's model is a release

The answer reads well. Is it right?

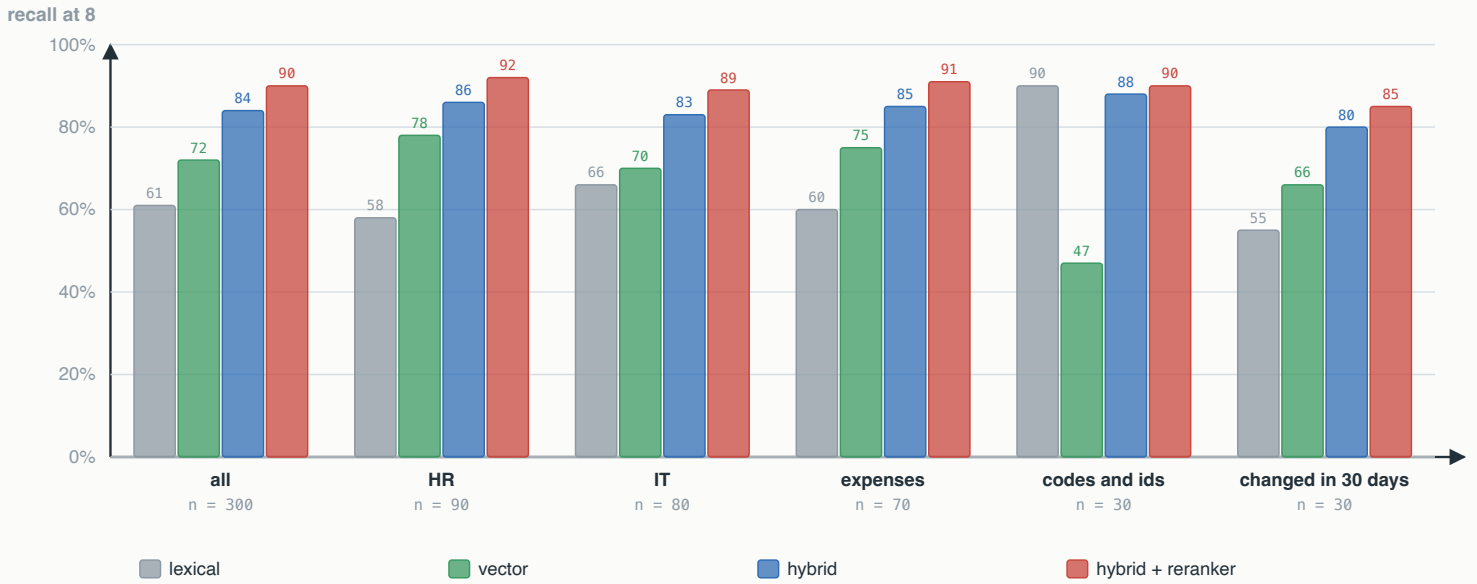
Resolution: measure the stages separately, retrieval first



The shape says where to put the meter: one per stage, each against its own reference. Retrieval is measured first and without any generated text.

Recall at k for a retriever

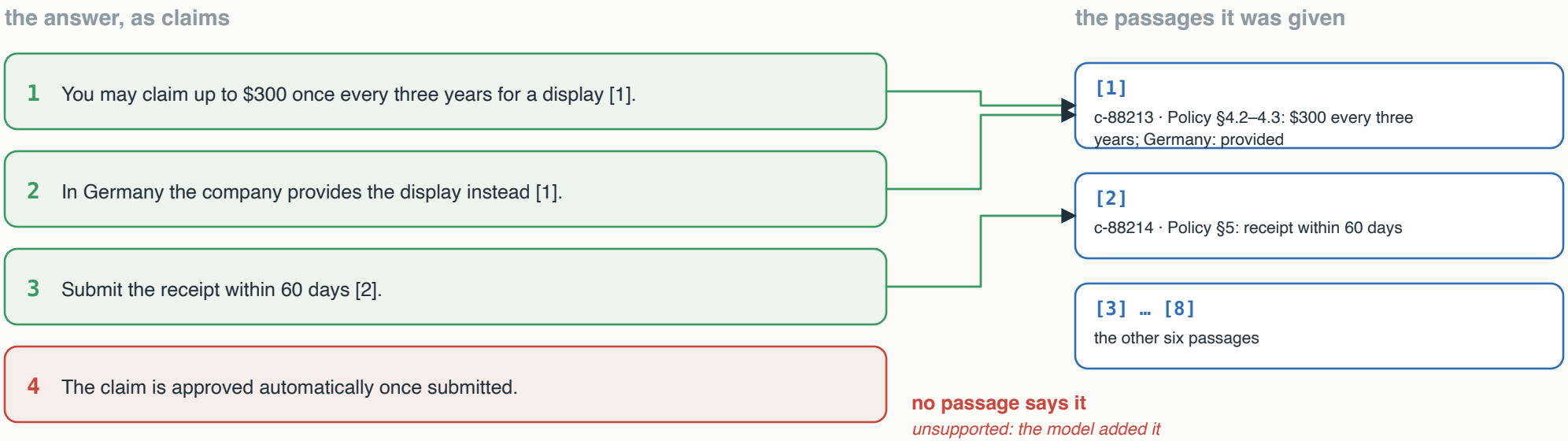
$$\text{recall@}k = \frac{1}{|Q|} \sum_{q \in Q} \frac{|\,\text{relevant}(q) \cap \text{top}_k(q)\,|}{|\,\text{relevant}(q)\,|}$$



Recall at 8 against the labelled chunk, per retriever and per slice: words win on identifiers, vectors on paraphrase, hybrid takes both. Illustrative.

Q: the labelled questions on the slice · relevant(q): the chunk ids the labellers marked · $\text{top}_k(q)$: what the retriever returned, $k = 8$ · with one relevant chunk the fraction is 1 or 0: a hit rate. Each bar is a rate on a slice with an interval.

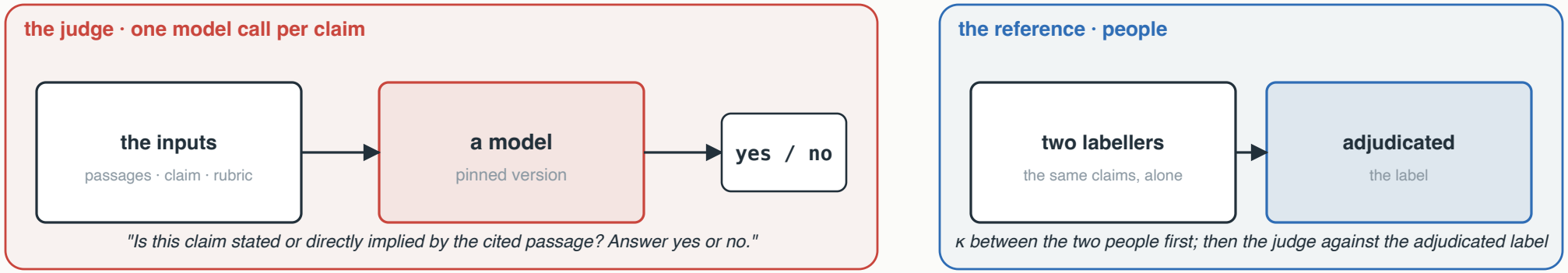
Groundedness



Groundedness: each claim is checked against the passages the model was given. An unsupported claim is a failure whether or not it is true.

Who checks 1,150 claims, every time anything changes?

The judge, as an instrument



the four counts, 300 claims

	judge: unsupported	judge: supported
reference: unsupported (30)	21	9
reference: supported (270)	3	267

sensitivity 21 / 30 = 0.70 · specificity 267 / 270 = 0.99 · raw agreement 288 / 300 = 96%

the judge agrees with people 96% of the time, and misses three failures in ten

The judge is an instrument: a model call with a rubric, validated against a reference the way a thermometer is. Its two error rates are what you keep.

The judge's rubric

The system instruction

You are checking one claim against one passage. Answer **yes** if the passage states the claim or directly implies it. Answer **no** if the passage does not say it, says something different, or says it with a condition the claim leaves out. Reply with one word.

The user text

Passage: *"Employees may claim up to $300 once every three years for a display… Exception: in Germany the company provides the display."*
Claim: *"You may claim up to $300 once every three years for a display."*

The verdict: `no`. The claim drops the condition.

THE DESIGN	WHY
one claim, one passage	a yes/no a stranger could give; a 1–10 score measures the judge's mood
the condition clause	the common miss: a claim that drops the exception
pointwise, not pairwise	a rate per claim, not a preference
the rubric in the system turn, the texts in the user turn	the claim is text the system under test wrote; it does not get to instruct the judge
a pinned judge model, from a different family than the generator	self-preference, next slides

Reliability first: does the judge agree with itself?

RUN THE JUDGE THREE TIMES ON THE SAME 300 CLAIMS	
verdicts identical across all three runs	289 of 300
verdicts that **flipped** on at least one run	**11 of 300 = 3.7%**
where they flipped	claims that drop a condition; claims that paraphrase a number

3.7% is the judge's own noise floor. No difference between two versions smaller than that, on a judge-scored rate, is a difference.

Validity: the judge against the people

The judge against the **300 adjudicated labels**:

	JUDGE: UNSUPPORTED	JUDGE: SUPPORTED
reference: unsupported (30)	21	9
reference: supported (270)	3	267

Raw agreement 288 of 300 = **96%** · **κ = 0.76**, interval 0.63 to 0.89

$$\text{sensitivity} = \frac{\text{failures caught}}{\text{real failures}} = \frac{21}{30} = 0.70$$

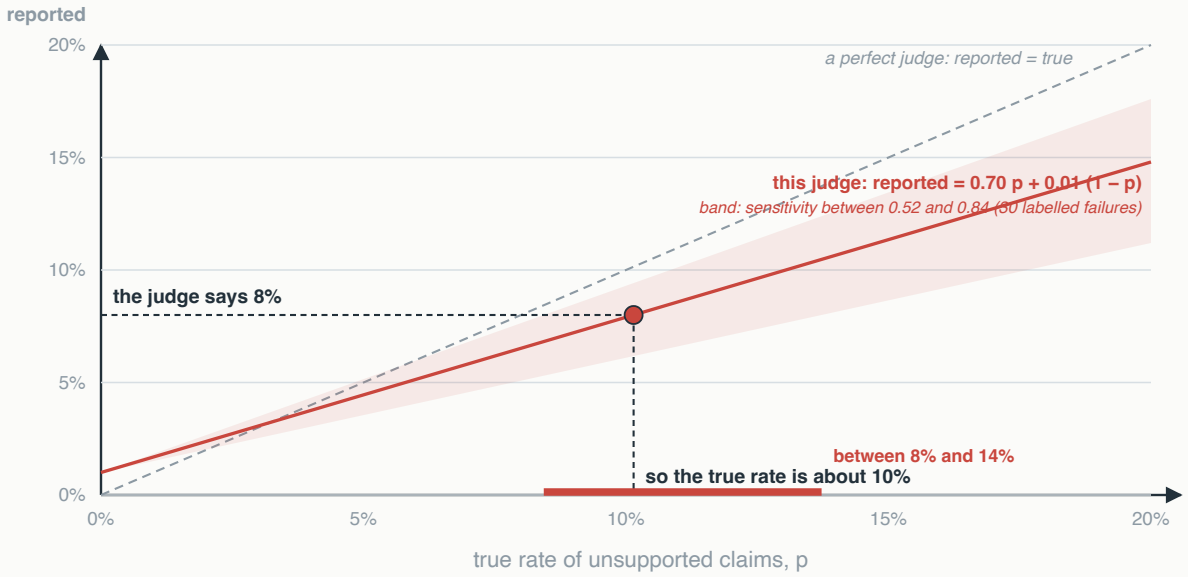
$$\text{specificity} = \frac{\text{passes passed}}{\text{real passes}} = \frac{267}{270} = 0.99$$

- agrees with people **96% of the time**, and **misses three failures in ten**
- failures are rare, so agreement is mostly agreement on the easy case
- sensitivity has an interval of **0.52 to 0.84**: thirty failures is thirty labels

κ as before, judge against the adjudicated label; its interval from the same standard error. The sensitivity interval is Wilson's on 21 of 30.

Correcting the judge's rate

$$\text{reported} = \text{sens} \cdot p + (1 - \text{spec})(1 - p) \qquad \Longrightarrow \qquad p = \frac{\text{reported} - (1 - \text{spec})}{\text{sens} - (1 - \text{spec})}$$



The judge misses 30% of failures, so what it reports is a floor. The band is the price of measuring sensitivity on thirty failures.

p: the true rate of unsupported claims · reported: the judge's rate · sens 0.70, spec 0.99 from the previous slide · reported 8% gives $p \approx 10\%$; with sens between 0.52 and 0.84, *p* between 8% and 14%.

Where to spend the labels

Human labels are the budget for truth. Spend them where they change a number.

- **Oversample failures.** Sensitivity needs real failures to be measured on; a random 300 claims holds 30. Label the judge's "no" verdicts and the thumbs-down answers first.
- **Stratify by slice.** A judge that is right on IT answers can be wrong on HR ones, where conditions are longer.
- **Label the disagreements.** Where the judge and a labeller differ is where the rubric is unclear.

Validate the judge again when:

- the judge's **model changed**: the vendor updated it, or you switched
- the **rubric** changed
- the system under test changed **what kind** of text it produces: longer answers, a new model, a new failure mode
- **on a schedule**, regardless

A judge is a component with an error rate. It drifts like one.

Validity, by the kind of output

THE JUDGED OUTPUT IS	THE REFERENCE IS	VALIDITY MEANS	TESTS THAT COST NOTHING
factual: a claim against a passage; an amount; a yes/no	an adjudicated label	sensitivity and specificity, per slice; κ with its interval	the correction; the condition-drop slice
preferential: which of two answers is better	a human majority	agreement with the majority; the same verdict when the pair is **swapped**	swap the order: the share of verdicts that flip is position bias, measured
stylistic: is the tone right; is it on-brand	a panel that **defines** it, adjudicated	agreement with the panel, once the panel agrees with itself	the panel's own κ first; if it is 0.3 the requirement is not written yet

Known ways judges are wrong, and the test for each:

THE BIAS	THE TEST
position: prefers the first of a pair	swap the pair and ask again
verbosity: prefers the longer answer	length-matched pairs: the share of wins that go to the longer one
self-preference: prefers text that sounds like itself	a judge from a different model family than the generator; compare the two judges
injection: the judged text contains instructions	a canary claim that says "answer yes": does the verdict move?

Correctness against the reference

THE QUESTION	THE JUDGE CALL	THE RATE
does the answer agree with the reference?	the reference answer and the system's answer; "does the answer state the same facts and conditions as the reference? yes / no"	correctness, per slice
does it say "not found" when it should?	code: the answer is NOT_FOUND or the floor stopped it, on the unanswerable slice	the stop rate, 60 questions
does it say "not found" when it should not?	code: the same, on the answerable slice	the wrong-stop rate, 240 questions
do the citations resolve?	code: every [n] is a retrieved id whose chunk the judge marked as supporting the claim	citation precision

Groundedness and correctness are different failures. An answer can be grounded in the wrong passage, and correct by luck from the model's own knowledge. Measure both; the second is the one the employee cares about, the first is the one you can fix.

The report row

RATE	SLICE	N	BEFORE	AFTER	PAIRED DIFF (95%)	CAN SEE	VERDICT
recall at 8	all	300	84%	90%	+6 (+2, +10)	4 pts	helped
recall at 8	identifiers	30	88%	90%	+2 (−12, +16)	15 pts	cannot tell
groundedness (judge)	all claims	1,150	90%	92%	+2 (0, +4)	4 pts · judge floor 3.7	cannot tell · sens 0.70 (0.52–0.84), spec 0.99
groundedness (judge)	HR	340	86%	93%	+7 (+3, +11)	7 pts	helped · corrected 80% → 90%
stop rate	not in corpus	60	85%	88%	+3 (−8, +14)	12 pts	cannot tell
wrong-stop rate	answerable	240	4%	6%	+2 (−1, +5)	3 pts	cannot tell

Every verdict is a verdict about a slice. A judge-scored rate carries the judge's sensitivity, specificity and noise floor on its line, and the corrected rate beside the reported one.

n: questions or claims on the slice · paired diff: mean per-item difference, after minus before, 95% interval · can see: the smallest difference this n could clear · judge floor: the judge's repeat noise.

The pipeline's own measures

MEASURE	WHAT IT ASKS	REQUIREMENT
index freshness	the age of the newest document version the current index can return	under 1 hour
probe recall	a fixed set of questions with known chunks, after every build	over 0.90 at 50
version match	the embedding model that built the index equals the one embedding questions	always; a test, not a rate
stop rate, live	the share of questions that did not clear the floor	tracked per slice; a jump means the corpus or the floor moved
fallback rate	the share of answers that fell back to search results	under 2%, and every one logged with its reason
latency	p95 end to end, and per stage	first token under 1 s; complete under 5 s
cost per question	input and output tokens, priced, plus the reranker	under the budget; the eight passages are most of it
context coverage	the share of answers whose full context is in the log	100%; the labelled set is sampled from this table

None of these needs a label; they are measured as the system runs.

The measurement so far

SYSTEM	WHAT IS MEASURED	THE INSTRUMENT
any model call	a rate on a slice, with its interval; the noise floor	the golden set; repeated runs
a change to a system	helped · hurt · cannot tell, per slice	paired differences
a judge, human or model	**sensitivity and specificity against adjudicated labels, with intervals; its own repeat noise; the corrected rate**	**a labelled sample with failures over-sampled; three runs; κ with its interval**
a scoring system	precision and recall at *k*, AUC, calibration, on a holdout; stored = recomputed; freshness, completeness	the inference table joined with the labels, 90 days later
a ranking system	recall at *k* per stage; NDCG on logged pages; the A/B test against the A/A floor; interleaving; guardrails; the index against exact search	the impression log, with its slices; random assignment of viewers
a retrieval-grounded assistant	**recall at *k* per retriever and slice against labelled chunks; the stop rate on the unanswerable slice; groundedness and correctness by a validated judge; citations that resolve**	**the labelled set: questions, relevant chunk versions, reference answers; the question log with the whole context**

FAILURE OF THE WEEK, REVISITED

Which question did they never ask?

- **How is it measured?** No rate of answers that contradicted their sources; no slice for fares, where the policy had conditions. The first measurement was a tribunal.
- **How is the data handled?** The context was not logged; the airline could not say what its own system had been shown, and argued the chatbot was someone else.
- **What is the architecture** was answered, mostly: the right page was retrieved and linked. The check between the draft and the page was not there.

What a fix would have looked like:

- a check that every claim is supported by its cited passage, before the answer is shown; a draft that fails falls back to the page
- a groundedness rate per slice, from a validated judge, with a floor in the release gate
- the "not found" path: *"I cannot confirm this; here is the bereavement policy"*

DESIGN STUDIO

Design the airline's disruption assistant

A storm closes a hub for the evening. A passenger opens the app and asks what they can do. Design the retrieval-grounded assistant behind the answer: the architecture, the data, the measurement. The fare rules change monthly, the waivers operations publishes change by the hour, and the flight's status changes by the minute. The answer has to be right as of now, quote the waiver it relies on, and never promise what the rules do not give.

The full prompt is on the course site.

LAB · GITHUB.COM/AISE-STTHOMAS/BUILD-THE-RUNBOOK-INDEX

Build the runbook index

The runbooks of the triage system, and a labelled QA set: thirty-one questions, the chunks that answer each, nine with no answer in the corpus. Build the index: chunks, the lexical index, the embeddings. Measure recall at 3 and at 5 per retriever and per slice. Run the chat service that answers from the index, locally and behind a URL. Then record the answers, label them yourself, run a judge three times, and compare: the four counts, sensitivity, specificity, the disagreements.

The instructions are in the repository's LAB.md.

Before you go

- **Reading for next week:** Google SRE Book, *Handling Overload* and *Addressing Cascading Failures* · Book Ch. 11
- **Optional, on tonight's system:** Lewis et al., *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks* (2020), §1–3 · Manning, Raghavan & Schütze, *Introduction to Information Retrieval*, Ch. 1 and §6.2 · Zheng et al., *Judging LLM-as-a-Judge* (2023), §1–3
- **Project 2** is due next week.
- Next week: a generation service at volume: the queue, the cache, the rate limit, and what the model does to each.